

第 15 章: The Documentation Interlude (文档插曲)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是第三部分(语句与语法)的收尾章, 主题从"写代码"短暂转向"给代码写文档"。它系统介绍了 Python 的文档生态: # 注释、docstring (文档字符串)、dir() 函数、help() 函数、Pydoc 的 HTML 界面、Sphinx、标准手册与网络资源——同时作为本部分最后一章, 还附上"常见编码陷阱"清单和第三部分的综合练习。

15.1 开篇引言 (Chapter Introduction)

英文原文

This part of the book concludes with a brief look at techniques and tools used for documenting Python code. Although Python code is comparatively readable by itself, a few well-placed and human-accessible comments can do much to help others understand the workings of your programs—especially when code grows larger than most in this book. As you'll see, Python includes both syntax and tools to make document

ation easier. In particular, the Pydoc system covered here can render a code file's documentation as either plain text in an interactive REPL, or HTML in a browser. While this topic is partly tools related, it's presented here because it both involves Python's syntax model and provides a resource for readers struggling to understand Python's toolset. For the latter purpose, this chapter also expands on documentation pointers first given in Chapter 4. As usual, because this chapter closes out its part, it also ends with some warnings about common pitfalls and a set of exercises for this part of the text, in addition to its chapter quiz.

中文翻译

本书的这一部分以一个简短的探索收尾：了解用于给 Python 代码写文档的技术和工具。虽然 Python 代码本身相对易读，但恰到好处、人能看懂的注释能极大地帮助他人理解你程序的运行机制——尤其是当代码变得比本书中的大多数例子更大时。正如你将看到的，Python 既提供了语法，也提供了工具，让文档编写更容易。特别是本章要讲的 Pydoc 系统，可以把代码文件的文档渲染成交互式 REPL 中的纯文本，或浏览器中的 HTML。虽然这个话题部分与工具相关，但把它放在这里，是因为它既涉及 Python 的语法模型，也为那些正在努力理解 Python 工具集的读者提供了资源。出于后一个目的，本章还扩充了第 4 章首次给出的文档指引。与往常一样，因为本章是一个部分的收尾章，除了本章测验外，它最后还附有关于常见陷阱的一些警告，以及本部分的练习题。

深度理解

·核心概念：本章不教新的核心语法（只有 docstring 算半个语法特性），而是教“怎么查资料、怎么被查”——它把 Python 的文档体系分成两条线：给自己代码写文档（# 注释、docstring），和查阅已有工具的文档（dir()、help()、Pydoc、手册、网站）。

·为什么把“文档”放在语句部分结尾：作者是有意安排的——前 14 章已经让你能写完整的程序，代码开始变大，此时“文档能力”正是刚需；同时本章作为第三部分最后一章，天然适合做“陷阱回顾 + 练习总复习”。

·Pydoc 是全书第一次深入接触的“自省式”工具：它读的是代码运行时对象身上的数据（doc 属性、函数签名）而非源码文本。这为后面讲“对象模型”埋下伏笔——文档信息竟然能作为数据存在对象里，这也是很多新手第一次意识到“Python 万物皆对象”的威力。

·生活类比：# 注释像贴在代码旁的便签纸，docstring 像商品上的铭牌，help() 像自动售货机上的“按此查看说明”按钮——同一件商品，查阅方式不同，详尽程度也不同。

15.2 Python 文档资源 (Python Documentation Sources)

英文原文

By this point in the book, you're probably starting to

realize that Python comes with an amazing amount of prebuilt functionality—built-in functions and exceptions, predefined object attributes and methods, standard-library modules, and more. And we've really only scratched the surface of each of these categories.

One of the first questions that bewildered beginners often ask is, How do I find information on all the built-in tools? This section provides tips on the various documentation sources available in Python. It also presents documentation strings (docstrings, for short), and the Pydoc system shipped with Python that makes use of them. These topics are somewhat peripheral to the core language itself, but they become essential knowledge as soon as your code becomes large or complex enough to challenge its readers—including yourself, six months down the road.

As summarized in Table 15-1, there are a variety of places to look for information about Python, with generally increasing verbosity. Because documentation is such a crucial tool in practical programming, we'll explore each of these categories in the sections that follow.

Table 15-1. Python documentation sources	
Form	Role
--- ---	# comments In-file documentation
The dir function	Lists of attributes available in objects
Docstrings: doc	In-file documentation attached to objects
Pydoc: the help function	Interactive hel

p for objects | | Pydoc: HTML reports | Module documentation in a browser | | Sphinx third-party tool | Richer documentation for larger projects | | The standard manuals | Official language and library descriptions | | Web resources | Online tutorials, examples, and so on |

中文翻译

读到本书这个位置，你可能已经开始意识到，Python 自带数量惊人的预制功能——内置函数和异常、预定义的对象属性和方法、标准库模块，等等。而且我们对每一个类别其实都还只是浅尝辄止。困惑的初学者最常问的问题之一就是：我怎么找到关于所有这些内置工具的信息？本节提供 Python 中各种可用文档来源的技巧，还介绍了文档字符串（简称 docstring）以及 Python 随附的、利用它们的 Pydoc 系统。这些主题对核心语言本身来说有点边缘，但一旦你的代码变得足够大、足够复杂，以至于对读者构成挑战——包括六个月后的你自己——它们就会变成必备知识。如表 15-1 所总结的，寻找 Python 信息的地方五花八门，详尽程度一般由浅入深。因为文档在实际编程中是如此关键的工具，我们将在接下来的各节中逐一探索这些类别。表 15-1. Python 文档来源 | 形式 | 作用 | |---|---| | # 注释 | 文件内文档 | | dir 函数 | 列出对象中可用的属性 | | 文档字符串：doc | 附着在对象上的文件内文档 | | Pydoc: help 函数 | 对象的交互式帮助 | | Pydoc: HTML 报告 | 浏览器中的模块文档 | | Sphinx 第三方工具 | 面向大型项目的更丰富文档 | | 标准手册 | 官方的语言与库描述 | | 网络资源 | 在线教程、示例等 |

深度理解

- 核心概念：这张表是 Python 文档体系的"地图"——从上到下，信息量越来越大：`#` 注释只存在于源码里，`dir()` 只给名字，`docstring` 给一段描述，`help()` 组装成完整报告，HTML 报告带排版和搜索，手册是权威参考，网络资源无穷无尽。层次越高，越适合"查证"；层次越低，越贴近"写作"。
- "包括六个月后的你自己"：作者这句调侃是文档的终极理由——写代码时你什么都记得，六个月后你和陌生人一样。文档的本质是"给未来的自己留一条记忆通道"。
- 初学者误区：有人以为"文档能力"不重要，先跳过本章。但实际上 `help()` 是每个 Python 程序员每天用的工具——它也是"自学能力"的起点：能自己查文档，就不必事事问人。
- 设计原因：Python 把"文档"做进语法 (`docstring`) 而不是仅仅依赖外部工具，因为 Guido 信奉"显式优于隐式"——代码旁边就该有说明，且要能被程序自动提取。

15.3 # 注释 (# Comments)

英文原文

As we've learned—and used in most example listings so far—hash-mark comments are the most basic way to document your code. Python simply ignores all the text following a `#` (as long as it's not inside a string

literal), so you can follow this character with any words and descriptions meaningful to programmers. Such comments are accessible only in your source files, though; to code comments that are more widely available, you'll use docstrings (ahead). In fact, current best practice (generally accepted convention) dictates that docstrings are best for larger functional documentation (e.g., "my script does this"), and # comments are best limited to smaller code documentation (e.g., "this strange expression does that") and are best limited in scope to a statement or small group of statements within a script or function. Docstrings are a broader topic we'll get to in a moment; first, let's see how to explore objects.

中文翻译

正如我们学过的——而且到目前为止大多数示例列表都在用——井号 (hash-mark) 注释是记录代码最基本的方式。Python 会直接忽略 # 后面的所有文本（只要它不在字符串字面量内部），所以你可以在这个字符后面写上任何对程序员有意义的词句和描述。不过，这类注释只能在你自己的源文件中看到；若要写适用范围更广的代码注释，就要用 docstring 了（稍后讲到）。事实上，当前的最佳实践（被普遍接受的惯例）规定：docstring 最适合较大规模的功能性文档（例如“我的脚本做这件事”），而 # 注释最好局限于较小规模的代码说明（例如“这个奇怪的表达式做那件事”），其范围最好限制在脚本或函数中的一条语句或一小群语句。docstring 是个更宏大的话题，我们一会儿就会讲到；

首先，让我们看看如何探索对象。

深度理解

·核心概念：`#` 注释是“就地”注释——解释器在词法分析阶段就把 `#` 到行尾的内容当作空白丢弃，完全不影响字节码。注意书里强调“只要不在字符串字面量内”：“`abc # def`”里的 `#` 是字符串内容，不是注释。

·最佳实践的分工：`docstring` 回答“这是什么”（面向使用者），`#` 注释回答“这里为什么这么写”（面向维护者）。这是社区约定（可追溯至 PEP 8 和 PEP 257 的精神），不是语法强制。

·生活类比：`#` 注释像课文里的页边批注，`docstring` 像每章的“本章概述”——批注属于某一行，概述属于整个章节。

·初学者误区：① 用 `#` 写整段功能说明——程序运行时别人根本看不到它；② 注释废话（“`x = 1 # 把 1 赋给 x`”——注释应该解释“为什么”，不是复述“是什么”。

15.4 `dir` 函数 (The `dir` Function)

英文原文

As we've also seen, the built-in `dir` function is an easy and automatic way to list all of the attributes available inside an object (i.e., its methods and simpler data items). It can be called with no arguments to list variables in the caller's scope, something the next part of

this book will define in full.

More usefully, it can also be called with any object that has attributes, including imported modules and objects of built-in types, as well as the name of a data type itself. For example, to find out what's available in a module, such as the standard library's `sys`, import it and pass it to `dir`:

```
$ python3 >> import sys >> dir(sys) [... many names here ...]
```

The list of attribute name strings you'll get back is omitted here because it's not small and is prone to vary per Python version; run this on your own for a better look.

In fact, there are currently 110 attributes in `sys`, though we generally care only about the 97 that do not have leading double underscores (two underscores usually means interpreter-related), or the 83 that have no leading underscore at all (one underscore usually means implementation private, informally). Rooting out such numbers is a prime example of the preceding chapter's list comprehension at work:

```
>> len(dir(sys)) # Number names in sys 110 >> len([x for x in dir(sys) if not x.startswith('__')]) # Non X names only 97 >> len([x for x in dir(sys) if not x[0] == '_']) # Ignore X names too 83
```

To find out what attributes are provided in objects of

f built-in types, run `dir` on a literal, an existing instance, or the name of the desired type. For example, to see all text-string attributes, you can pass an empty or the type name `str`:

```
>> dir("") ['add', ... more names here ..., 'zfill'] >> dir(str) == dir("") True
```

The `dir` result for any built-in type includes a set of attributes that are related to the implementation of that type (mostly, for expression operators). Much as in modules, they all begin and end with double underscores to make them distinct, and you can safely ignore them at this point in the book (they'll resurface later for OOP). For instance, there are 81 text-string attributes, but only 47 that correspond to named methods or data; lists have even fewer:

```
>> len(dir("")), len([x for x in dir("") if not x.startswith('_')]) (81, 47) >> len(dir([])), len([x for x in dir([]) if not x.startswith('_')]) (48, 11)
```

To list names, filter out double-underscored items that are not of common program interest by running the same list comprehensions, but print the attributes. To demo, here are the named attributes in lists and dictionaries:

```
>> [a for a in dir(list) if not a.startswith('_')] ['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort'] >> [a for a in dir(dict) if not a.startswith('_')] ['clear', 'copy', 'fromkeys', 'get', 'items', 'keys',
```

```
pop','popitem','setdefault','update','values']
```

This may seem like a lot to type to get an attribute list, but beginning in the next chapter, you'll learn how to wrap such code in a simple, importable, and reusable function so you don't need to type it again.

Aside: type names like `str`, `list`, and `dict` work in `dir` because they are actually names of types in Python today, not just type-converter functions; calling one of these invokes its constructor to generate an instance of that type. Part VI will have more to say about constructors and operator overloading methods when we discuss classes.

The `dir` function is mostly a memory jogger—it provides a list of attribute names, but it does not tell you anything about what those names mean. For such extra information, we need to move on to the next documentation source.

NOTE — Attributes in IDEs: Some GUIs for Python work, including IDLE, have features that list attributes on objects automatically, which can be viewed as alternatives to `dir`. IDLE, for example, will list an object's attributes in a pop-up selection window when you type a period after the object's name and pause or press Tab. This is mostly meant as an autocomplete feature, though, not an information source. Chapter 3 and Appendix A have introductory info on IDLE.

中文翻译

正如我们已经见过的，内置 `dir` 函数是列出对象内所有可用属性（即它的方法和更简单的数据项）的一种简单而自动的方式。它可以不带参数调用，以列出调用者作用域中的变量——这是本书下一部分将完整定义的内容。更有用的是，它也可以接受任何带有属性的对象，包括导入的模块、内置类型的对象，以及数据类型本身的名字。例如，想了解 `sys` 这样的标准库模块里有什么，导入它再传给 `dir` 即可：`$ python3 >>> import sys >>> dir(sys) [...很多名字.....]` 你拿回来的属性名字字符串列表在这里被省略了，因为它的规模不小，而且容易随 Python 版本变化；最好自己运行一下看看。事实上，`sys` 目前有 110 个属性，但我们一般只关心其中 97 个不以双下划线开头的（两个下划线通常意味着与解释器相关），或者 83 个完全没有前导下划线的（一个下划线通常非正式地表示实现私有的）。挖出这类数字，正是上一章列表推导式（list comprehension）大显身手的好例子：`>>> len(dir(sys)) # 计算 sys 中的名字数量110 >>> len([x for x in dir(sys) if not x.startswith('_')]) # 只要非 X 名字97 >>> len([x for x in dir(sys) if not x[0] == '_']) # 连 X 名字也忽略83` 想找出内置类型的对象提供了哪些属性，可以对一个字面量、一个现有实例、或目标类型的名字运行 `dir`。例如，要看字符串的全部属性，可以传一个空字符串或类型名 `str`：`>>> dir('') ['add', ...更多名字....., 'zfill'] >>> dir(str) == dir('') True` 任何内置类型的 `dir` 结果都包含一组与该类型实现相关的属性（大多是表达式运算符）。与模块类似，它们都以双下划线开头和结尾以示区别，在本书当前阶段你可以安全地忽略它们（OOP 部分它

们会再次出现)。例如，字符串有 81 个属性，但只有 47 个对应命名的 (named) 方法或数据；列表的更少：

```
>>> len(dir('')), len([x for x in dir('') if not x.startswith('__')]) (81, 47)
>>> len(dir([])), len([x for x in dir([]) if not x.startswith('__')]) (48, 11)
```

要列出名字，运行同样的列表推导式，过滤掉双下划线的、一般程序不感兴趣的项目，但这次打印属性。作为演示，下面是列表和字典中“有名有姓”的属性：

```
>>> [a for a in dir(list) if not a.startswith('__')] ['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
>>> [a for a in dir(dict) if not a.startswith('__')] ['clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

为了得到一个属性列表要打这么多字，看起来挺费劲，但从下一章开始，你会学会把这类代码包进一个简单、可导入、可复用的函数里，这样就不用再重复输入了。附注 (Aside)：str、list、dict 这类类型名能用于 dir，是因为它们在今天的 Python 中其实是类型的名字，而不只是类型转换函数；调用其中一个会调用它的构造器 (constructor) 来生成该类型的一个实例。第六部分讨论类时，会对构造器和运算符重载方法有更多说明。dir 函数主要是个“记忆辅助器” (memory jogger) ——它给出属性名的列表，但不告诉你这些名字是什么意思。要获取这类额外信息，我们需要进入下一个文档来源。注意——IDE 中的属性 (Attributes in IDEs)：一些 Python 的 GUI 环境 (包括 IDLE) 有自动列出对象属性的功能，可以看作 dir 的替代品。例如，IDLE 在你输入对象名后的句点并暂停或按 Tab 键时，会在弹出选择窗口中列出对象的属性。不过这主要是一个自动补全 (autocomplete) 功能，而不是信息源。第 3 章和附录 A 有

深度理解

·核心概念：`dir()` 返回的是属性名的字符串列表，它做的是“目录索引”而不是“内容解释”。三个级别：`dir()`（当前作用域的变量）→ `dir(对象)`（对象的属性）→ 对类型名调用（该类型实例共有的属性）。

·底层实现：`dir()` 的实现本质是调用对象的 `dir()` 方法（若无则回退到 `dict + 类型属性扫描`）。它列出的东西包括：实例属性、类属性、继承来的属性。你看到的 `add`、`len` 这类双下划线属性，是解释器让对象支持 `+`、`len()` 等运算符的内部钩子——这为第六部分 OOP 埋下伏笔。

·为什么按版本变化：属性列表来自 Python 的 C 实现（C Python 的内置类型），每版本都可能增删内部属性，所以书里给出的数字（110/97/83、81/47、48/11）只是写作时的快照。不要背这些数字，要背方法。

·列表推导式在这里的作用：`[x for x in dir(sys) if not x.startswith('_)]` 是一个“过滤管道”——把无关的机器内部名字剔除。这是上一章学到的推导式最典型的实战用途之一：处理集合数据时优先想到推导式。

·初学者误区：① 以为 `dir()` 会给出方法的用法说明——不，它只给名字；② 试图用 `dir()` 查模块时忘记先 `import`——没导入的对象根本不存在；③ 把 `dir('')` 与 `dir('abc')` 当成不同结果——其实字符串类型相同，结果相同（书里用 `dir(str) == dir('')` 验证了这一点）。

·生活类比：dir() 像超市货架上的商品分类标签（列出所有商品名），但不告诉你每件商品怎么用——想知道用法得看说明书（docstring），而 help() 是自动帮你翻说明书。

代码分析

```
>>> import sys                                # sys
>>> dir(sys)                                   # sys

# dir(sys) sys __dir__
# __dict__

>>> len(dir(sys))                             #
110
>>> len([x for x in dir(sys) if not x.startswith('__')]) # __X...
97
>>> len([x for x in dir(sys) if not x[0] == '_'])      # _X ...
83

# 110 > 97 > 83 sys 13 __X__
# 14 _X ""
# 83

>>> dir('')                                   # str
['__add__', '...', 'zfill']
>>> dir(str) == dir('')                       #
True

# str '' —
# ""

>>> len(dir('')), len([x for x in dir('') if not x.startswith('__')...
(81, 47)                                     # 81 47 ""
>>> len(dir([])), len([x for x in dir([]) if not x.startswith('__')...
(48, 11)                                     # 48 11
```

```
>>> [a for a in dir(list) if not a.startswith('__')]
['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', ...
 'remove', 'reverse', 'sort']
>>> [a for a in dir(dict) if not a.startswith('__')]
['clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popi...
 'setdefault', 'update', 'values']
```

·解释器如何处理：dir(list) 不需要创建任何实例——类型本身就是对象，直接传即可。内部的"列出属性"其实是遍历 PyObject 类型结构（C 层的 tpdict）以及其继承链，把名字收集成列表。

·数字背后的信息：字符串 81 个属性中 34 个是双下划线（add、contains 等运算符协议方法），列表 48 个中 37 个是双下划线。可见运算符重载协议是内置类型"内存"的大头——这些正是第六部分类要学的魔法方法。

15.5 文档字符串与 doc (Docstrings and doc)

英文原文

Besides # comments, Python supports documentation that is automatically attached to objects and retained at runtime for inspection. Syntactically, such comments are coded as string literals of any type, except bytes and f-strings. They are located at the tops of code components (module files and function and class statements), and before any other executable code (# comments, including Unix-style #! lines, are OK before

e them). When present, the text of such strings, known as docstrings, is automatically stuffed into the doc attributes of the corresponding objects.

中文翻译

除了 # 注释, Python 还支持自动附着到对象上、并在运行时保留供检视 (inspection) 的文档。从语法上讲, 这种注释被写成任意类型的字符串字面量——但 bytes 和 f-string 除外。它们位于代码组件 (模块文件、函数和类语句) 的顶部, 且必须在任何其他可执行代码之前 (# 注释, 包括 Unix 风格的 #! 行, 可以放在它们前面)。当存在时, 这类字符串的文本——称为 docstring——会被自动塞进相应对象的 doc 属性中。

深度理解

·核心概念: docstring 是有语法位置的字符串——Python 编译器在解析 def/class/module 时, 若第一个语句 (除注释外) 是一个字符串字面量, 就把它编译成该对象的文档, 存入 doc 属性。它同时是"注释"和"运行时可访问的数据"。

·为什么排除 bytes 和 f-string: bytes 不是给人读的文本; f-string 是"带求值"的字符串——docstring 必须是编译期可静态确定的常量文本, f-string 的内容在运行时才计算, 无法可靠地作为文档存储。这是 Python 3.12 之前的经典限制, 也正是书里示例用 r'...' 和普通字符串的原因。

·"自动"二字的关键: 你不需要写任何赋值语句——只要

字符串出现在正确位置，解释器自动完成“存入 doc”这一动作。这是“约定优于配置”（convention over configuration）的早期体现。

·生活类比：docstring 像公司入职时贴在工位上的“岗位说明书”——你入职（定义对象）时人事自动贴好（编译器自动写入 doc），任何人路过（运行时）都能查看。

15.5.1 用户自定义 docstring (User-defined docstrings)

英文原文

For example, consider the file `docstrings.py` in Example 15-1. You can safely ignore most of its code here (we'll study functions, modules, and classes in the next three parts of this book), but notice its three string literals: because they are coded at the beginning of the file, and at the start of a function (`def`) and class (`class`) within it, they are taken as docstrings and are saved in the associated object's doc when this file is loaded by a run or import.

Example 15-1. `docstrings.py`

```
python """Module documentation This module defines a name, function and class.
```

```
"""
```

```
edition = 6 def square(x): r"""Function documentation Returns the \square\ of its \numeric\ argument.
```

'''

```
return x 2 # power operator class PartVI: "Class documentation for \U0001F40D "
```

pass # Top-level code print(square(edition)) print(square.doc) print(PartVI.doc) Some fine points about this example's docstrings: Literals: the first of this file's docstrings uses a triple-quoted block at the top of the file. This is common for docstrings as it allows for multiline comments, but any sort of text string (except f-strings) will work. Both single- or double-quoted one-liners like that in the class are fine, but don't readily support multiline text.

Raw strings: as the function demos, r'...' strings work too—and may even be required to suppress unwanted backslash escapes and avoid a syntax warning (and eventual error!) for unrecognized escapes each time code is run or recompiled to bytecode (see Chapter 7's coverage of new \ deprecations in Python 3.12).

Content: docstrings can contain any sort of text, including the class's Unicode escape and raw emoji (per the Unicode intro in Chapter 4 and the whole story in Chapter 37). Notice that the \U Unicode escape requires backslashes to be used, so it precludes using raw strings to avoid syntax errors in the future (use \\\).

The whole point of this documentation protocol is that your comments are retained for inspection in doc attributes after the file is loaded. Thus, to display the

docstrings associated with the module and the two code components it defines, we simply import the file and print their doc attributes, where Python has saved the text. Assuming this module is located in the directory in which we're currently working (and deferring to Chapter 3's note about imports and directories for background info on why that matters):

```
$ python3 >> import docstrings 36 Function documentation Returns the \square\ of its \numeric\ argument. Class documentation for >> print(docstrings.doc) Module documentation This module defines a name , function and class.
```

```
>> print(docstrings.square.doc) Function documentation Returns the \square\ of its \numeric\ argument.
```

```
>> print(docstrings.PartVI.doc) Class documentation for
```

You will usually want to use `print` like this to view docstrings; otherwise, you'll get a single string with embedded `\n` newline characters. Also note that `import` runs the file's top-level prints (hence the output immediately following the import), as does launching it as a top-level script. Within the file, `square` is just `square`, a simple variable:

```
$ python3 docstrings.py 36 Function documentation Returns the \square\ of its \numeric\ argument. Class documentation for
```

You can also attach docstrings to methods of classes (covered in Part VI too), but because, as you'll learn, these are just def statements nested in class statements, they're not a special case. For instance, to fetch the docstring of a method function inside a class within a module, you would simply extend the path to go through the class: `module.class.method.doc`. We'll code an example of method docstrings in Chapter 29.

NOTE — Docstrings futurism: If you look closely, you'll notice that multiline docstrings, like that of `square`, retain their leading indentation as it was in the code. Python 3.13, still on the drawing board as this note is being written, plans to remove leading indentation from docstrings, mostly to save space in bytecode files and memory. Even before this mod, though, you won't normally see the indents in docs, because tools like `Pydoc` already strip and reformat, as you'll learn shortly.

中文翻译

例如，看看示例 15-1 中的文件 `docstrings.py`。这里你可以放心忽略它的大部分代码（我们会在本书接下来的三个部分研究函数、模块和类），但请注意它的三个字符串字面量：因为它们被写在文件的起始处，以及文件内一个函数（`def`）和一个类（`class`）的开头，所以它们被视为 docstring，并在该文件被运行或导入加载时，保存到相关对象的 doc 中。示例 15-1. `docstrings.py` `python` `""" Module documentation This module defines a name, function and`

```
class. """ edition = 6 def square(x): r'''Function docu
mentation Returns the \square\ of its \numeric\ argu
ment.'''return x 2 # power operator class PartVI: "Clas
s documentation for \U0001F40D " pass # Top-level c
ode print(square(edition)) print(square.doc) print(Par
tVI.doc)
```

关于这个示例的 docstring，有几个值得注意的细节：

- 字面量 (Literals)：** 这个文件的第一个 docstring 在文件顶部用了三引号 (triple-quoted) 块。这对 docstring 很常见，因为它支持多行文本，但任何类型的文本字符串 (f-string 除外) 都可以。像类里那种单引号或双引号的单行字符串也没问题，只是不好支持多行文本。
- 原始字符串 (Raw strings)：** 正如函数演示的，`r'...'` 字符串也能用——甚至可能是必需的：它可以抑制不想要的转义、避免每次运行或重新编译成字节码时出现无法识别转义的语法警告 (乃至最终报错！) (见第 7 章关于 Python 3.12 中新弃用的警告的讲解)。
- 内容 (Content)：** docstring 可以包含任何文本，包括类的 Unicode 转义和原始 emoji (见第 4 章的 Unicode 入门和第 37 章的完整故事)。注意 `\U` Unicode 转义本身需要使用反斜杠，所以它排除了用原始字符串来避免未来语法错误的可能 (要用 `\\`)。这套文档协议的要点在于：文件加载后，你的注释会被保留在 `doc` 属性中供检视。因此，要显示与模块及其定义的两个代码组件相关联的 docstring，我们只需导入该文件并打印它们的 `doc` 属性——Python 已把文本保存在那里。假设该模块位于我们当前工作的目录中 (为什么要紧，请参见第 3 章关于导入与目录的说明)：

```
$ python3 >>> import docstrings
36 Function documentation Returns the \square\ of its \numeric\ argument. Class documentation for >>>
```

`print(docstrings.doc)` Module documentation This module defines a name, function and class. `>>> print(docstrings.square.doc)` Function documentation Returns the `\square\` of its `\numeric\` argument. `>>> print(docstrings.PartVI.doc)` Class documentation for 你通常会用 `print` 来查看 docstring; 否则, 你会得到一个嵌着 `\n` 换行符的单个字符串。还要注意, `import` 会运行文件的顶层打印语句 (所以紧跟 `import` 之后就有输出), 把它当作顶层脚本启动也是如此。在文件内部, `square` 就是 `square`, 一个简单的变量: `$ python3 docstrings.py 36` Function documentation Returns the `\square\` of its `\numeric\` argument. Class documentation for 你也可以把 docstring 附着到类的方法 (method) 上 (也是第六部分的内容), 但正如你将学到的, 方法不过是嵌套在 class 语句里的 `def` 语句, 所以它们不是特例。例如, 要取模块内某个类中某个方法函数的 docstring, 只需把路径延长穿过类即可: `module.class.method.doc`。我们会在第 29 章写一个方法 docstring 的示例。注意——docstring 的未来 (Docstrings futurism): 如果仔细观察, 你会发现像 `square` 这样的多行 docstring 会原样保留代码里的前导缩进。写这条注记时还在酝酿中的 Python 3.13 计划移除 docstring 的前导缩进, 主要是为了节省字节码文件和内存的空间。不过, 即使在这个改动之前, 你通常也不会在文档里看到这些缩进, 因为 `Pydoc` 之类的工具会剥离并重新排版——你稍后就会学到。

深度理解

·核心概念: docstring 的三个位置 = 三种对象: 文件顶

部 → 模块的 doc; def 块首部 → 函数的 doc; class 块首部 → 类的 doc。规则只有一条：组件定义体中的第一个语句是字符串字面量（允许 # 注释和 #! 行打头），它就自动成为 docstring。

·底层实现：CPython 编译器（compile() 阶段）解析到函数/类/模块体时，检查第一条语句；若是字符串常量（const），则把它作为 doc 存入该对象的属性，同时不生成任何“执行该字符串”的字节码——它被消耗掉了，不再作为普通表达式求值。这也是为什么它在运行时“消失”而出现在 doc 里。

·原始字符串为什么“可能必需”：Python 3.12 开始对未知转义序列（如 \s）发 SyntaxWarning，未来版本会升级为 SyntaxError。docstring 里若有 \square 这样的文字，普通字符串会把 \s 当转义；用 r'...' 则原样保留反斜杠——示例中 r''' 正是这个用途。

·\U0001F40D 的细节：\U 转义（8 位十六进制 Unicode 码点）必须写在普通字符串里才能生效，原始字符串中它是字面文字；若想在原始字符串里写真实反斜杠要用 \\。这是书里特意展示的“raw 字符串不等于万能”的边界情况。

·导入会执行顶层代码：import docstrings 后立刻看到 36、Function documentation 等输出，是因为导入会运行文件的全部顶层语句（第三部分的 print 也是顶层语句）。这是第 3 章的知识，也是后文“Pydoc 查看脚本文档会运行脚本”的伏笔。

·为什么要用 `print` 而不是直接敲变量名：REPL 直接求值 `square.doc` 会显示带 `\n` 转义的原始字符串；`print()` 则把转义渲染成真实换行，可读。这是新手常忽略的 REPL 显示差异。

·缩进的保留与剥离：多行 `docstring` 原样保留代码缩进（存进 `doc` 时不去缩进），工具（`Pydoc`、`inspect.getdoc`）负责剥离。Python 3.13 后编译器层面会做类似处理以省空间。

·初学者误区：① 把 `docstring` 写在 `def` 后面第二、第三行——只有“第一个语句”才算数；② 以为 `docstring` 必须是三引号——单行双引号也行；③ 忘记 `docstring` 的读取路径——模块是 `mod.doc`，函数是 `mod.func.doc`，方法要一路穿到 `mod.Class.method.doc`。

代码分析

```
"""
Module documentation
This module defines a name, function and class.
"""

edition = 6                                #

def square(x):
    """
    Function documentation
    Returns the \square\ of its \numeric\ argument.
    """
    return x ** 2                            # x

class PartVI:
    "Class documentation for \U0001F40D "    # \U0001F40D Unico...
    pass
```

```
# Top-level codeimport
print(square(edition))          # 366
print(square.__doc__)           # docstring
print(PartVI.__doc__)           # docstring
```

·解释器如何处理：

1. 编译阶段：编译 docstrings.py 时，模块体的第一个语句是三引号字符串 → 存入 docstrings.doc；编译 square 函数体时，第一个语句是 r'''...''' → 存入 square.doc；编译 PartVI 类体时，第一个语句是 "..." 单行字符串 → 存入 PartVI.doc。三个字符串都被"消耗"，不会生成求值字节码。

2. 运行时：edition = 6 给模块命名空间添加变量；def square 创建函数对象（其 doc 已就位）；class PartVI 创建类对象。最后的三个 print 依次取用 doc 属性并打印。

3. 注意 print(square.doc) 与直接在 REPL 敲 square.doc 的区别：前者渲染换行，后者显示带 \n 的原始字符串。

·设计思想：docstring 与普通注释的关键区别——注释被编译器丢弃，docstring 是"可编程访问的文档"。这让工具（help、Pydoc、Sphinx、doctest）都能用统一接口 obj.doc 取文档，形成了 Python 文档生态的基石。

·运行方式影响输出：python3 docstrings.py（顶层脚本）与 import docstrings（模块导入）都会执行顶层 print，输出相同——因为模块在两种情况下都是"从第一行执行到最后一行"。

15.5.2 Docstring 规范 (Docstring standards)

英文原文

As mentioned earlier, common practice today recommends hash-mark comments for only smaller-scale documentation about an expression, statement, or small group of statements. Docstrings are better used for higher-level and broader functional documentation for a file, function, or class, and have become an expected part of Python software. Beyond these guidelines, though, you still must decide what to write. Although some companies have internal standards, there is no broad consensus about what should go into the text of a docstring. There have been various markup language and template proposals, including HTML, but they don't seem to have caught on in the Python world. Frankly, convincing programmers to document their work using handcoded HTML probably won't happen in our lifetimes, but this shouldn't apply to documentation in general. Documentation tends to have a lower priority than it should. Too often, if you get any docs in a file at all, you count yourself lucky (and even better if they're accurate and up to date). While technically optional, documentation is a crucial part of well-written programs, and you're encouraged to comment your code liberally. When you do, though, there is no standard docstring format; if you want to use them, anything goes today. Just as for writing code it

self, it's up to you to create docstring content and keep it up to date, but common sense is probably your best ally in both tasks.

中文翻译

如前所述，如今的常见做法建议：`#` 注释只用于较小规模的、针对某条表达式、语句或一小群语句的文档；docstring 更适合用作文件、函数或类的高层次、较宽泛的功能性文档，它已经成为 Python 软件中可预期的组成部分。不过，除了这些指导原则，写什么内容仍然要由你自己决定。虽然一些公司有内部标准，但关于 docstring 文本里应该写什么，并没有广泛的共识。曾经有过各种标记语言和模板提案，包括 HTML，但它们似乎在 Python 世界没有流行起来。坦率地说，在我们有生之年，说服程序员用手写 HTML 来记录自己的工作大概不会发生——不过这不适用于一般意义上的文档。文档的优先级往往低于它应有的水平。太常见了：一个文件里只要能有点文档，你就该谢天谢地了（要是还准确、还跟得上代码，那就更好了）。虽然从技术上说是可选的，但文档是编写良好的程序中至关重要的一部分，我们鼓励你大量注释自己的代码。不过，当你真要写时，并没有标准的 docstring 格式；如今你爱怎么用就怎么用。就像写代码本身一样，创建 docstring 内容并保持更新是你的责任，而在两项任务中，常识大概是你最好的盟友。

深度理解

·核心概念：这段讲的是社区惯例而非语法强制——“docstring 管大、注释管小”是推荐分工；docstring 里写什么，没有强制标准（宽松性：anything goes）。这是 Python

on 社区"文档自由"文化的一部分。

- PEP 257 是什么：作者没有点名，但官方对 docstring 约定的权威出处是 PEP 257 (Docstring Conventions, 2001 年)。它规定：docstring 用三引号 `"""` 包裹；首行是简洁的摘要句（一句话概括用途，以句号结尾）；多行 docstring 摘要后空一行，再写详细说明；函数 docstring 应说明参数、返回值、异常。PEP 257 约束的是格式约定，与书里说的"内容无共识"并不矛盾——写什么内容仍是你的自由。

- 为什么"用手写 HTML 写文档"行不通：HTML 标签噪音大、可读性差、难维护——所以 Sphinx 选择了 reStructuredText (后文 15.8 节) 这种"接近纯文本"的轻量标记。这是"文档语言必须比代码语言简单"的教训。

- "文档优先级低"是行业普遍现象：软件工程里有句老话："代码被阅读的次数远多于被编写的次数"，而文档是被阅读者最先看的东西。Lutz 的建议朴素而实用：用常识判断，写自己六个月后能看懂的文本。

- 初学者误区：① 以为 docstring 有"必须遵守"的标准格式，看到别人用不同风格就纠结——没有；② 把 docstring 写成复述代码的流水账——好的 docstring 说"用途与契约"（输入、输出、副作用），不说"实现细节"；③ 写完 docstring 就再也不更新——过时文档比没有文档更危险。

- 现代实践补充：如今大型项目普遍采用 docstring 模板（如 Google style、NumPy style），并为参数类型加注解（type hints），配合 mypy 等工具做静态检查——但这

些都是社区约定，不是语言强制。

15.5.3 内置 docstring (Built-in docstrings)

英文原文

While it's important to document our own code, it turns out that built-in modules and objects in Python use similar techniques to attach documentation above and beyond the attribute lists returned by `dir`. For example, to view a human-readable description of a built-in module, import it and print its doc string—as we did for our own code:

```
>> import sys >> print(sys.doc) This module provides access to some objects used or maintained by the interpreter and to functions that interact strongly with the interpreter. Dynamic objects: argv -- command line arguments; argv[0] is the script pathname if known path -- module search path; path[0] is the script directory, else "modules" -- dictionary of loaded modules ... more text omitted ...
```

For finer-grained details, functions, classes, and methods within built-in modules have attached descriptions in their doc attributes as well:

```
>> print(sys.getrefcount.doc) Return the reference count of object. The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to getrefcount(
```

).

You can also read about built-in functions via their docstrings:

```
>> print(map.doc) map(func, iterables) --> map object
Make an iterator that computes the function using arguments from each of the iterables. Stops when the shortest iterable is exhausted.
```

In fact, you can get a wealth of information about built-in tools by inspecting their docstrings this way—but you don't have to. The Python help function largely automates this for you, as the next section will explain.

中文翻译

给自己代码写文档固然重要，但事实证明，Python 的内置模块和对象也用类似的技术来附加文档——它们提供的信息远超 `dir` 返回的属性列表。例如，要查看某个内置模块的人类可读描述，导入它并打印它的 `doc` 字符串即可——就像我们对自己的代码做的那样：

```
>>> import sys
>>> print(sys.doc) This module provides access to some objects used or maintained by the interpreter and to functions that interact strongly with the interpreter. Dynamic objects: argv -- command line arguments; argv[0] is the script pathname if known path -- module search path; path[0] is the script directory, else "modules" -- dictionary of loaded modules...更多文本省略.....要更细粒度的细节，内置模块中的函数、类和方法在它们的 doc
```

属性里也带有描述：`>>> print(sys.getrefcount.doc)` Return the reference count of object. The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to `getrefcount()`. 你也可以通过 `docstring` 了解内置函数：`>>> print(map.doc)` `map(func, iterables) --> map object` Make an iterator that computes the function using arguments from each of the iterables. Stops when the shortest iterable is exhausted. 事实上，通过这种方式检视它们的 `docstring`，你能获得大量关于内置工具的信息——但你不需要亲自动手。Python 的 `help` 函数会为你大规模地自动化这件事，下一节会解释。

深度理解

- 核心概念：`docstring` 机制是统一的——你自己的模块用，CPython 的内置模块（C 语言写的 `sys`、`map` 等）也用。`sys.getrefcount.doc` 说明即使是 C 实现的函数，其文档也挂在 `doc` 属性上，对 Python 程序员完全透明。
- 底层实现：CPython 的 C 代码里，每个内置函数/模块都有一个文档字符串常量（如 `PyDocSTR(...)` 宏定义），在创建对应对象时填进 `doc`。所以 `doc` 访问路径对纯 Python 对象和 C 内置对象完全一致——这是“统一对象模型”的又一体现。
- `sys.getrefcount` 的文档本身就是教程：它提醒你“返回值通常比你预期的高 1，因为把参数传给 `getrefcount` 本身算一次临时引用”——引用计数是 CPython 内存管理的基础，第五部分会深入。

·信息层次: `dir()` 给名字 → `doc` 给描述 → `help()` 给排版好的报告。这三者递进, 就像"目录 → 词条 → 整本手册"。

·初学者误区: ① 以为内置对象没有文档, 只能上网查——其实文档就在对象身上; ② 直接敲 `sys.doc` 而不加 `print`, 看到一堆 `\n` 就以为坏了。

15.6 Pydoc: `help` 函数 (Pydoc: The `help` Function)

英文原文

The docstring technique proved to be so useful that Python eventually added a tool that makes docstrings even easier to display. Namely, Pydoc is Python code that knows how to extract docstrings and associated structural information and format them into nicely arranged reports of various types. Additional tools for extracting and formatting docstrings are available in the open source domain (including third-party tools that may support structured text—search the Web for pointers), but Python ships with Pydoc in its always-present standard library. There are a variety of ways to launch Pydoc, but the two most prominent are the built-in `help` function and the Pydoc browser-based interface. Of these, `help` may be the most straightforward to use and is always available in a REPL. We met

help briefly in Chapter 4. It invokes Pydoc to generate a simple plain-text report for any Python object. In this mode, help text looks much like a "manpage" on Unix-like systems. In fact, outside GUIs like IDLE, help text works the same way as a Unix "more" when there are multiple pages of text—press the space bar to move to the next page, Enter (or your keyboard's equivalent) to go to the next line, and Q to quit the display. GUIs generally scroll help text instead. Let's turn to a few examples to see how this works.

中文翻译

docstring 技术被证明如此有用，以至于 Python 最终添加了一个让 docstring 更容易显示的工具。它就是 Pydoc——一段知道如何提取 docstring 及相关结构信息、并把它们排版成各种类型漂亮报告的 Python 代码。开源领域还有更多提取和排版 docstring 的工具（包括支持结构化文本的第三方工具——上网搜索即可找到线索），但 Python 在它永远随身的标准库中自带了 Pydoc。启动 Pydoc 的方式有多种，但最主要的是两种：内置 help 函数和 Pydoc 的浏览器界面。其中，help 可能是最直接易用的，而且在 REPL 中随时可用。我们在第 4 章已经简单见过 help。它为任何 Python 对象调用 Pydoc 生成一份简单的纯文本报告。在这种模式下，帮助文本看起来很像 Unix 类系统上的 "manpage"（手册页）。事实上，在 IDLE 这类 GUI 之外，当帮助文本有多页时，它的翻页方式与 Unix 的 "more" 命令一致——按空格键翻到下一页，按回车（或键盘上的等价键）到下一行，按 Q 退出显示。GUI 环境一般直接滚

动显示帮助文本。下面看几个例子，看看它是怎么工作的。

深度理解

- 核心概念：help() 是 Pydoc 的"前台"，Pydoc 是标准库 pydoc 模块的"后台"。help(obj) = 让 Pydoc 提取 obj 的 doc + 结构信息（签名、类层次、数据成员）→ 排版成报告 → 送进分页器显示。help 的底层就是 pydoc 模块（import pydoc 可见），它是用 Python 自己写的。
- 底层实现（help 的完整机制）：内置的 help 是 site 模块启动时注册的一个 Helper 实例。它拿到对象后调用 pydoc.renderdoc()：① 用 inspect 模块获取对象的类型与签名（inspect.signature/inspect.getdoc）；② inspect.getdoc 会取 doc 并做 textwrap.dedent（剥离公共前导缩进）和清理；③ 对模块/类还会遍历 dir() 结果分类整理（FUNCTIONS、DATA、CLASSES 等章节）；④ 输出交给分页器（Unix 的 more/less 或 Windows 的 more.com）。所以你在 help() 报告里看到的结构，是你也能用 dir()/inspect 自己拿到的——Pydoc 只是"拼装工"。
- 为什么 help 显示有分页：UNIX 哲学——长文本用管道交给分页器，按需翻页，不刷屏。空格翻页、回车逐行、Q 退出，正是 more 的交互约定。
- 生活类比：help() 像"点菜时叫服务员来讲解"——服务员（Pydoc）手里有完整的菜单资料（doc），还知道每道菜的构成（签名、参数），用统一话术讲给你听。

15.6.1 对内置工具运行 help (Running help on built-in tools)

英文原文

Because built-in tools come with the docstrings we saw earlier, help gives us a level of reference docs for Python, midway between dir lists and the full manuals. Here it is live, reporting on a standard-library module's function:

```
>> import sys >> help(sys.getrefcount) Help on built-in function getrefcount in module sys: getrefcount(object, /) Return the reference count of object. The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to getrefcount().
```

You do not have to import sys in order to call help, but you generally have to import sys to get help on sys this way, because help expects an object reference to be passed in. Alternatively, you can also get help for a module you have not imported by quoting the module's name as a string. For example, help('sys') and help('sys.getrefcount') both work too, without the import step.

For larger objects such as modules and classes, the help display is broken down into multiple sections, the preambles of which are shown here. Run this interactively to see the full report:

```
>> help(sys) Help on built-in module sys: NAME sys
MODULE REFERENCE https://docs.python.org/3.12/library/sys.html ... more omitted ... DESCRIPTION This
module provides access to some objects used or maintained by the ... more omitted ... SUBMODULES monitoring
FUNCTIONS
```

```
breakpointhook = breakpointhook(...)... more omitted ... DATA stderr = <io.TextIOWrapper name='<stderr>'
mode='w' encoding='utf... ... more omitted ... FILE (built-in)
```

Some of the information in this report is docstrings, and some of it (e.g., function call patterns) is structural information that Pydoc gleans automatically by inspecting objects' internals using tools available to your code too.

Besides modules, you can also use help on built-in functions, methods, and types. To get help for a built-in type, try either the type name (e.g., dict for dictionary or str for string); an actual object of the type (e.g., {} or ''); or a method of an actual object or type name (e.g., 's'.join or str.join).¹ For an entire type, you'll get a large display that describes all the methods available for that type; for a method, the help is more focused:

```
>> help(dict) Help on class dict in module builtins:
class dict(object) | dict() -> new empty dictionary | dict(mapping) -> new dictionary initialized from a map
```

ping object's | ... more omitted ... >> help(str.replace)
Help on methoddescriptor: replace(self, old, new, count=-1, /) Return a copy with all occurrences of substring old replaced by new. ... more omitted ... >> help(''.replace) Help on built-in function replace: replace(old, new, count=-1, /) method of builtins.str instance Return a copy with all occurrences of substring old replaced by new. ... more omitted ... >> help(ord) Help on built-in function ord in module builtins: ord(c, /) Return the Unicode code point for a one-character string.

The "/" in function docs means that arguments before it must be passed by position only (not by name), but this is substantially above this chapter's pay grade. See the next part of this book for details on this function syntax.

¹ Note that asking for help on an actual string object directly (e.g., help('xyz')) doesn't work as you may expect: you usually get no help, because strings are interpreted specially—as a request for help on a topic or tool by name, as described earlier. You must use the str type name in this context, though both an empty string (help('')) and string method names referenced through actual objects (help('').join)) work fine. An interactive help mode, started by typing just help(), avoids manual help calls, and may avoid some of their drama.

中文翻译

因为内置工具自带我们前面见过的 docstring, help 为我们提供了一级 Python 参考文档——介于 dir 列表和完整手册之间。下面是它的实况, 报告标准库模块的一个函数: >>> import sys >>> help(sys.getrefcount) Help on built-in function getrefcount in module sys: getrefcount(object, /) Return the reference count of object. The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to getrefcount(). 调用 help 本身不需要导入 sys, 但要这样获取关于 sys 的帮助, 一般必须先 import sys, 因为 help 期望传入一个对象引用。另一种办法: 把模块名写成字符串传给 help, 就不必导入。例如 help('sys') 和 help('sys.getrefcount') 都能工作, 无需 import。对模块和类这类较大的对象, help 显示会拆成多个小节, 这里展示的是各小节的引言部分。要看到完整报告, 请交互式运行: >>> help(sys) Help on built-in module sys: NAME sys MODULE REFERENCE <https://docs.python.org/3.12/library/sys.html>...更多省略.....DESCRIPTION This module provides access to some objects used or maintained by the...更多省略.....SUBMODULES monitoring FUNCTIONS breakpointhook = breakpointhook(...)...更多省略.....DATA stderr = <io.TextIOWrapper name='<stderr>' mode='w' encoding='utf'.....更多省略...FILE (built-in) 这份报告里有些信息来自 docstring, 有些 (例如函数调用形态) 则是 Pydoc 通过检视对象内部自动收集的结构信息——它用的工具, 你的代码同样能用。

除了模块，你也可以对内置函数、方法和类型使用 help。要获取内置类型的帮助，可以用类型名（如字典用 dict、字符串用 str）；该类型的实际对象（如 {} 或 ''）；或某个实际对象或类型名的方法（如 's'.join 或 str.join）。¹ 对整个类型，你会得到一份巨大的显示，描述该类型可用的全部方法；对单个方法，帮助则更聚焦：

```
>>> help(dict) Help on class dict in module builtins: class dict(object) | dict() -> new empty dictionary | dict(mapping) -> new dictionary initialized from a mapping object's | ...更多省略.....>>> help(str.replace) Help on methoddescriptor: replace(self, old, new, count=-1, /) Return a copy with all occurrences of substring old replaced by new. ...更多省略.....>>> help('').replace) Help on built-in function replace: replace(old, new, count=-1, /) method of builtins.str instance Return a copy with all occurrences of substring old replaced by new....更多省略.....>>> help(ord) Help on built-in function ord in module builtins: ord(c, /) Return the Unicode code point for a one-character string. 函数文档里的 "/" 表示它之前的参数必须只按位置传递（不能按名字传），但这远超本章的难度范围。这种函数语法的细节见本书下一部分。1 注意：直接对实际的字符串对象请求帮助（如 help('xyz'）不会像你预期的那样工作：你通常得不到帮助，因为字符串被特殊解释了——被当作“按名字请求某个主题或工具的帮助”，如前文所述。在这个语境里你必须用 str 类型名；不过空字符串（help('')）和通过实际对象引用的字符串方法名（help(''.join)）都能正常工作。只输入 help() 启动的交互式帮助模式，可以避免手动调用 help，也许还能避开它的这些小坑
```


深度理解

·核心概念: `help()` 有"三级输入": ① 传对象 (`help(sys.getrefcount)`) ——需要先导入; ② 传字符串名 (`help('sys')`) ——免导入, Pydoc 内部帮你 import; ③ 传交互模式 (`help()`) ——进入 `>>>` 模式的帮助解释器, 在里面直接输入名字。三者调用同一条 Pydoc 流水线。

·为什么 `help('xyz')` 不工作: 这是字符串的特殊解释——Pydoc 把字符串当"主题/工具名"处理, 'xyz' 不是合法名字所以无结果。而 `help('')` 空字符串是特例 (被视为请求 `str` 类型帮助)。同理, `help('sys')` 能工作是因为 `sys` 是真实模块名。这是新手最容易踩的坑之一。

·报告的章节结构是自动分类的: NAME/DESCRIPTION/FUNCTIONS/DATA/CLASSES/FILE——Pydoc 遍历模块的 `dir()`, 按属性类型 (函数对象→FUNCTIONS, 类→CLASSES, 其他→DATA) 分桶。这再次说明: 你看到的 report 结构 = 你也能用 `dir() + type() + doc` 自己拼出来的东西, Pydoc 只是排版工。

·/ 与位置参数: 签名 `getrefcount(object, /)` 中 / 是 PEP 570 (Python 3.8) 引入的"位置限定参数"标记——/ 之前的参数不能用关键字=值传参。`ord(c, /)` 同理。书里说"超出本章难度"是给读者减压, 你可以先眼熟它, 第 16 章学函数定义时会遇到。

·"pay grade" (能力层级) 的说法: Lutz 用薪酬等级打比方——这个知识点现在不用懂, 学到函数部分自然明白

- 教学上这叫"概念锚点": 先挂个名字, 以后详细展开。
- 帮助文本与 manpage: help 的排版风格 (NAME/DESCRIPTION 等大写小节标题) 刻意模仿 Unix man 手册——因为 Python 诞生于 Unix 文化, pydoc 最初就是替代 "man python" 的文档方案。
- 生活类比: help() 像咨询台: 你可以报"商品编号" (对象)、报"商品名" (字符串名), 或者直接走进咨询室 (交互模式) 连续问; 但报一个不存在的东西 ('xyz'), 服务员只能摊手。

15.6.2 对自有代码运行 help (Running help on your own code)

英文原文

Finally, the help function works just as well on your modules as it does on built-ins. Here it is reporting on components defined in the docstrings.py file we coded earlier in Example 15-1. Again, some of this is our docstrings, and some is information automatically extracted by inspecting objects' structures:

```
>> import docstrings >> help(docstrings.square)
Help on function square in module docstrings:
square(x)
Function documentation Returns the \square\ of its \
numeric\ argument.
```

```
>> help(docstrings.PartVI)
Help on class PartVI in module docstrings:
```

```
class PartVI(builtins.object) | Class documentation for  
| | Data descriptors defined here: | ... more omitted ...
```

And asking for help on the entire module collects all parts at once, with your docstrings in the mix:

```
>> help(docstrings) Help on module docstrings: NA  
ME docstrings DESCRIPTION Module documentation  
This module defines a name, function and class. CLAS  
SES builtins.object PartVI
```

```
class PartVI(builtins.object) | Class documentation for  
| | Data descriptors defined here: | ... more omitted ...  
FUNCTIONS square(x) Function documentation Retur  
ns the \square\ of its \numeric\ argument. DATA
```

```
edition = 6 FILE /Users/me/MY-STUFF/Books/Dev/6E  
/LP6E/LP6E/Examples/Chapter15/docstrings.py
```

If you look closely, you'll notice that we didn't get our file's printed output on import this time, just because it was imported earlier; as noted in Chapter 3, imports run just once per file per session (more in Part V).

中文翻译

最后，help 函数对待你自己的模块和对待内置对象一样好。下面是它对我们前面在示例 15-1 里编写的 docstrings.py 文件中各组件的报告。同样，有些内容是我们写的 docstring，有些则是通过检视对象结构自动提取的信息：>>> import docstrings >>> help(docstrings.square) Help on function square in module docstrings: square(x) F

unction documentation Returns the \square\ of its \n
numeric\ argument. >>> help(docstrings.PartVI) Help
on class PartVI in module docstrings: class PartVI(buil
tins.object) | Class documentation for | | Data descrip
tors defined here: | ...更多省略.....对整个模块请求帮助，
则一次性汇集所有部分，你的 docstring 也在其中：>>>
help(docstrings) Help on module docstrings: NAME d
ocstrings DESCRIPTION Module documentation This
module defines a name, function and class. CLASSES
builtins.object PartVI class PartVI(builtins.object) | Cla
ss documentation for | | Data descriptors defined her
e: | ...更多省略.....FUNCTIONS square(x) Function docu
mentation Returns the \square\ of its \numeric\ argu
ment. DATA edition = 6 FILE /Users/me/MY-STUFF/B
ooks/Dev/6E/LP6E/LP6E/Examples/Chapter15/docstri
ngs.py 如果仔细看，你会发现这次导入时没有看到我们文
件里打印的输出——只是因为它在前面已经被导入过了；
如第 3 章所述，每个会话中每个文件的导入只执行一次（
第五部分有更多说明）。

深度理解

· 核心概念：help() 对"自己写的代码"和"内置代码"一视同仁——因为机制相同：都读 doc + 结构检视。你从第一行 docstring 开始，就已经在为你的模块免费生成"manpage"了。

· square(x) 这一行从哪来：函数签名不是 docstring 内容，而是 Pydoc 用 inspect.signature 从函数对象上自动提取的。你的 def square(x) 定义决定了它显示 square(x

)。这就是"结构信息自动提取"的含义。

- `class PartVI(builtins.object)` 的含义：类名后的括号显示继承关系——`PartVI` 继承自 `builtins.object`（所有类的根）。`Data descriptors` 一节显示的是类属性里的描述符（如方法对象），第六部分详细讲。

- `FILE` 路径：报告底部显示的 `FILE` 是模块的源文件路径——`Pydoc` 通过模块对象的 `file` 属性获取。这提示你：`mod.file` 是找"这个模块代码在哪"的快捷方式（本书后文也用它定位 `pydoc.css`）。

- 模块只导入一次：第二次 `import docstrings` 不再重复执行顶层 `print`——Python 的模块缓存机制（`sys.modules`）保证同一会话内一个文件只加载一次。第五部分会深入。

- 初学者误区：① 以为 `help()` 只对内置对象有效——自己的模块同样受益，前提是写了 `docstring`（没有 `docstring` 时 `help` 就只剩签名，形同 `dir`）；② 以为 `help()` 会执行你的模块代码——`import` 时执行，`help()` 本身只读取已加载的对象。

15.7 Pydoc: HTML 报告 (Pydoc: HTML Reports)

英文原文

The text displays of the help function are adequate in many contexts, especially at the interactive prompt. T

o readers who've grown accustomed to richer presentation mediums, though, they may seem a bit rudimentary. This section presents the HTML-based flavor of Pydoc, which renders module documentation more graphically for viewing in a web browser, and can even open one automatically for you. This browser interface scheme combines both search and display in a web page that communicates with an automatically started local server.

中文翻译

help 函数的文本显示在许多场合都够用，尤其是在交互式提示符下。不过，对已习惯于更丰富呈现媒介的读者来说，它可能显得有点简陋。本节介绍 Pydoc 的 HTML 版形态：它把模块文档渲染得更具图形化，供 Web 浏览器查看，甚至能自动为你打开一个浏览器。这套浏览器界面方案把搜索和显示都整合在一个网页里，该网页与一个自动启动的本地服务器通信。

深度理解

·核心概念：Pydoc 的两种模式是同一引擎的两套外壳——纯文本报告（help 函数）和 HTML 报告（pydoc -b），数据都来自对象检视，只是渲染目标不同。HTML 版的价值在于：可点击的交叉链接、搜索框、常驻浏览器随时查阅。

·架构：pydoc -b 启动两个本地组件——文档服务器（监听 localhost 的随机空闲端口）和客户端（你的浏览器，

向服务器请求页面)。这与生产环境里的 Web 架构 (客户端-服务器) 同构, 只是全在本机。

·为什么用随机端口: localhost:53965 里的端口是"默认任意未占用端口"——避免与已运行的 Web 服务冲突。若想固定端口, 不带 -b 启动 Pydoc 即可看到更多选项。

·生活类比: help() 是打印在便签上的说明书, pydoc -b 是图书馆的检索终端——前者看单件物品, 后者在整座图书馆里浏览、搜索、翻页。

15.7.1 使用 Pydoc 的浏览器界面 (Using Pydoc's browser interface)

英文原文

To view docs in a browser, you'll usually launch Pydoc's main script with a `pydoc -b` command line. This spawns two components running locally on your device: a documentation server, and a web-browser client that provides both page display and search input for content fetched from the server. A command line like the following starts the show: `$ python3 -m pydoc -b`

Server ready at <http://localhost:53965/> Server commands: [b]rowser, [q]uit server> q

This command works on all PCs; it's known to also work on Android devices in the Pydroid 3 app's Terminal, but is spotty on phones in general. It uses the Python `-m` argument to locate Pydoc's module file on the module-import search path (covered in Part V of this book), and run it as

a top-level script. Per Appendix A and Chapter 3, use `py` instead of `python3` in this command on Windows. On macOS and Linux, a `pydoc3 -b` command has the same effect, and on Windows the "Module Docs" entry in Python's Start menu automatically runs Pydoc's `-b` mode. However you start it, Pydoc's browser mode opens with its index page, captured in Figure 15-1. The index page lists all the modules available for imports and includes search boxes (e.g., `Get` loads a named module's docs), as well as topic guides and reserved-word info. You'll see links to the docs of every module on your module search path, including the directory where Pydoc is launched. Pydoc's server runs on the local host and on a dedicated but by default arbitrary unused port, but this can be tailored; launch Pydoc without `-b` for more options. (Figure 15-1: The top-level index start page of the Pydoc browser interface)

When you click or tap on a module's entry in Pydoc's module-index page (or enter its name in `Get`), you get a page of docs for that module alone—which is essentially the same info that `help` provides in an interactive REPL, but formatted for display in your browser. Figure 15-2, for example, shows the page we get for the `docstrings` module we coded earlier in Example 15-1. It's the same report created for `help(docstrings)` with a simple HTML layout. (Figure 15-2: Pydoc browser interface displaying a user-defined module's docs)

Experiment with Pydoc on your own for more insi

ght. It's an optional tool, but provides extra docs for both built-in tools provided by Python, as well as doc string-loaded code files that you write yourself.

中文翻译

要在浏览器中查看文档，你通常用 `pydoc -b` 命令行启动 `Pydoc` 的主脚本。这会启动运行在你设备本地的两个组件：一个文档服务器，和一个 Web 浏览器客户端——后者为从服务器获取的内容提供页面显示和搜索输入。类似下面的命令行就开始了：`$ python3 -m pydoc -b` Server ready at `http://localhost:53965/` Server commands: `[b]rowser`, `[q]uit server`> `q` 这个命令在所有 PC 上都能用；已知它在安卓设备的 `Pydroid 3` 应用终端里也能工作，不过在手机上整体时灵时不灵。它用 Python 的 `-m` 参数在模块导入搜索路径上定位 `Pydoc` 的模块文件（本书第五部分会讲），并把它作为顶层脚本运行。按附录 A 和第 3 章的说明，Windows 上此命令用 `py` 而不是 `python3`。在 macOS 和 Linux 上，`pydoc3 -b` 命令有同样效果；Windows 上 Python 开始菜单里的 "Module Docs" 项会自动运行 `Pydoc` 的 `-b` 模式。无论你怎么启动，`Pydoc` 的浏览器模式都会以索引页 (index page) 打开，即图 15-1 所示。索引页列出所有可供导入的模块，包含搜索框（例如 `Get` 会加载指定模块的文档），以及主题指南和保留字信息。你会看到模块搜索路径上每个模块的文档链接，包括启动 `Pydoc` 的那个目录。`Pydoc` 的服务器运行在本地主机上，使用一个专用但默认任意的未占用端口；不过这是可定制的——不带 `-b` 启动 `Pydoc` 可看到更多选项。（图 15-1：Pydoc 浏览器界面的顶层索引起始页）当你在 `Pydoc` 模块索引页点击

某个模块的条目（或在 Get 框中输入它的名字），你会得到仅关于该模块的一页文档——它本质上与 help 在交互式 REPL 中提供的信息相同，只是排版成适合浏览器显示的样子。例如图 15-2 显示的是我们前面在示例 15-1 中编写的 docstrings 模块的页面，它就是 help(docstrings) 生成的同一份报告配上简单的 HTML 布局。（图 15-2: Pydoc 浏览器界面显示一个用户自定义模块的文档）建议你自己动手试试 Pydoc，以获得更多体会。它是可选工具，但为 Python 提供的内置工具、以及你自己编写的带 docstring 的代码文件，都提供了额外文档。

深度理解

- 核心概念：python3 -m pydoc -b 的 -m 参数是关键语法——"把 pydoc 这个模块当作脚本运行"。解释器会沿模块搜索路径（sys.path）找到 pydoc.py 并执行它。这与 python3 -m pip、-m http.server 是同一机制。
- 索引页 = 你的 sys.path 地图：索引页列出的每个模块都来自模块搜索路径——包括标准库目录和当前工作目录。这意味着你启动 Pydoc 的目录决定你"能看到哪些自写模块的文档"。这是理解"Pydoc 只能访问可导入文件夹中的文件"的基础（15.7.3 节还会强调）。
- server> 提示符是 Pydoc 自己的命令行：输入 b 打开浏览器、q 退出服务。它是个小小的交互式 REPL，只是换了个提示符。
- 平台差异：Windows 没有 python3 命令（用 py）、没有 pydoc3 命令（用开始菜单 "Module Docs"）——这是 Python 打包分发历史的遗留，第 3 章和附录 A 的细

节在此复用。

·初学者误区：① 直接双击运行 `pydoc.py` 而不带 `-b`——那只是显示命令行选项；② 在非工作目录启动 `pydoc -b` 后找不到自己的模块——因为 `Pydoc` 只能看到启动目录及 `PYTHONPATH` 里的东西；③ 忘记端口号是随机的——每次启动都可能不同，看终端输出的 URL。

15.7.2 定制 `Pydoc` (Customizing `Pydoc`)

英文原文

If you run this live, your `Pydoc` will probably look different than the screenshots in the book, because colors were customized on the machine used to capture the images. Though this is entirely optional and requires some web-development knowledge, you can customize `Pydoc`'s appearance, too, by editing the simple CSS file it uses to render its pages. To modify colors, edit file `pydoc.css` stored in the `pydocdata` folder alongside `Pydoc`'s module—which you can locate on any platform by importing `Pydoc`'s module and inspecting its file attribute: `$ python3 -c "import pydoc; print(pydoc.file)" ... foldepath .../pydoc.py` `$ ls ... foldepath .../pydocdata` # `Pydoc`'s CSS customization file is in here `init.py topics.py pycache pydoc.css` On Windows, use `py` and `dir` instead of `python3` and `ls`. This uses the Python `-c` command-line argument to submit code to be run as a string, which is handy for short bits of one-off code like this; here, the two statements separ

ated by a semicolon run the same as they would if they were typed one at a time at a REPL's `>>>` prompt. You can also find the path to Pydoc by running `help('pydoc')`: its path shows up at the bottom of the display. Once you've located the CSS file, edit it to customize as you like (but save the original first as a fallback). As an example, this book's captures use RGB color string `#173166` for index-decor. All of which assumes CSS skills beyond this book's scope (plus an undocumented config file that's prone to change or vanish in the future!), but is probably enough to get you started if you want to have a go.

中文翻译

如果你实际运行一下，你的 Pydoc 看起来大概和书里的截图不一样，因为截图机器的颜色被定制过。虽然这完全可选、而且需要一些 Web 开发知识，但你也可以通过编辑 Pydoc 渲染页面所用的简单 CSS 文件来定制它的外观。要修改颜色，编辑存储在 `pydocdata` 文件夹里的 `pydoc.css` 文件——它就在 Pydoc 模块旁边；在任何平台上，你都可以通过导入 Pydoc 模块并检视它的 `file` 属性来定位它：`$ python3 -c "import pydoc; print(pydoc.file)" ...路径.../pydoc.py` `$ ls ...路径.../pydocdata` # Pydoc 的 CSS 定制文件在这里面 `init.py` `topics.py` `pycache` `pydoc.css` 在 Windows 上，用 `py` 和 `dir` 代替 `python3` 和 `ls`。这里用到 Python 的 `-c` 命令行参数，把要运行的代码作为字符串提交——很适合这种一行的一次性代码；此处用分号分隔的两个语句，其运行效果与在 REPL 的 `>>>` 提示符下一句一句输入

相同。你也可以通过运行 `help('pydoc')` 找到 Pydoc 的路径：它的路径显示在显示内容的底部。一旦定位了 CSS 文件，就可以按喜好编辑定制（但先保存原件作回退）。举个例子，本书截图用 RGB 颜色字符串 `#173166` 作为 `index-decor` 的颜色。这一切都假定你具备超出本书范围的 CSS 技能（外加一个未文档化的、将来容易改变甚至消失的配置文件！），但如果你想试试，这些大概足够你起步了。

深度理解

- 核心概念：Pydoc 的页面外观由它自带的 CSS 文件 `pydoc.css` 控制——`file` 属性是定位“模块源代码/资源文件在哪”的万能钥匙：`import pydoc` 后 `pydoc.file` 给出 `pydoc.py` 的绝对路径，旁边的 `pydocdata` 目录存着页面资源

- `-c` 参数的本质：`python3 -c "代码字符串"` 让解释器把命令行里的字符串当模块体编译执行——注意字符串里是两条语句用分号连接，而 REPL 中分号的效果也是“一行内依次执行两条语句”，所以两者等价。这是“代码即数据、一行代码跑一次”的快捷方式。

- 为什么改颜色要先备份：修改标准库文件有风险——Python 升级可能覆盖你的改动，且该配置文件“未文档化、可能改变或消失”。作者的潜台词：定制有风险，动手需谨慎，功能优先于外观。

- web 开发知识门槛：CSS 是另一个领域。作者故意带过这个“劝退”话题——本章目标是“会用文档工具”，不是“美化文档工具”。

·生活类比: file 像"正品溯源二维码"——任何模块都能查到它出生(安装)的位置;改 pydoc.css 像给公开图书馆的书换封面——能改,但别人升级馆藏时你的封面会被换掉。

15.7.3 更多 Pydoc 技巧 (More Pydoc tips)

英文原文

Pydoc works by importing selected files to extract their documentation. This has two usage implications. First, Pydoc can access only files in folders included on the module search path used by imports. This path automatically includes Python's standard library, so you can view docs for all built-in modules. It also includes the current working directory—the folder from which Pydoc was started—but that may be meaningless when started from a Windows Start button. Change your PYTHONPATH setting to include other code folders as needed, per Appendix A. Second, although Pydoc can render docs for both importable modules and runnable scripts in accessible folders, script docs comes with a catch: when a file is selected for help, Pydoc must import it in order to collect its documentation, and as we learned in Chapter 3, importing runs a file's top-level code. Hence, script help implies a script run. Imported modules normally just define tools when run, so this is usually irrelevant. If you ask for the documentation of a top-level script file, though, the scrip

t will be run, and the console window where you launched Pydoc serves as the script's standard input and output for any user interaction. This may work better for some scripts and modes than others; script IO may appear before or after help is dismissed in console mode, before help is scrolled in a GUI like IDLE, or interleaved oddly with Pydoc's own server-command prompts in browser mode. Later in the book you'll learn ways to code top-level logic that's kicked off only when a file is run, not when it is imported (e.g., by nesting it under a test for variable name being 'main'). If scripts use these protocols, they're safe to view in Pydoc, because it won't run any top-level code as an inadvertent side effect of viewing docs. Finally, Pydoc has additional tools we'll skip here for space, including a -w switch for saving HTML docs to a file for later viewing, and a plain-text mode run from a command line with just a topic name that works the same as help in a REPL. Again, run Pydoc without any command-line arguments for its full set of options. NOTE — Blast from the past: Pydoc once had a simple GUI mode, invoked by flag -g, that launched a tkinter GUI client that communicated with the server. This was available until Python 3.2, when the current browser-only mode was deemed sufficient to warrant dropping the longstanding GUI mode in full. Browsers are GUIs too, of course, and the CSS configuration scheme for Pydoc that arose with the browser mode assumes extra

web-development knowledge that many Python users won't have. In open source, those with time and desire to change things define the future for everyone.

中文翻译

Pydoc 的工作方式是导入选中的文件来提取它们的文档。这带来两个使用层面的含义。第一，Pydoc 只能访问导入搜索路径 (module search path) 所含文件夹里的文件。这条路径自动包含 Python 的标准库，所以你可以查看所有内置模块的文档。它还包含当前工作目录——即启动 Pydoc 的那个文件夹——但从 Windows 开始菜单启动时，这个目录可能没什么意义。按附录 A 的说法，按需修改 PYTHONPATH 设置以包含其他代码文件夹。第二，虽然 Pydoc 既能渲染可访问文件夹中可导入的模块、也能渲染可运行的脚本的文档，但脚本文档有个坑：当一个文件被选中请求帮助时，Pydoc 必须先导入它以收集文档，而正如我们在第 3 章学过的，导入会运行文件的顶层代码。因此，查看脚本帮助就意味着脚本会被运行。导入的模块运行起来通常只是定义工具，所以这通常无关紧要。但如果你请求的是一个顶层脚本文件的文档，脚本就会被运行，你启动 Pydoc 的那个控制台窗口将充当脚本的标准输入输出，供任何用户交互使用。这对某些脚本和某些模式可能好些、对另一些就差些；脚本的 IO 在控制台模式下可能出现在帮助被关闭之前或之后，在 IDLE 这类 GUI 里出现在帮助滚动之前，或者在浏览器模式下与 Pydoc 自己的服务命令提示符奇怪地交错。本书后面你会学到编写顶层逻辑的方法，让它们只在文件被运行时触发、而导入时不触发（例如把它嵌套在变量 `name` 是否为 `'main'` 的判断下）。如果脚本使用这些协议，它

们在 Pydoc 中查看就是安全的，因为查看文档不会意外地触发任何顶层代码的副作用。最后，Pydoc 还有一些此处因篇幅而略过的工具，包括把 HTML 文档保存到文件以便日后查看的 `-w` 开关，以及一个纯文本模式——在命令行只给一个主题名即可运行，效果与 REPL 里的 `help` 相同。再说一次，不带任何命令行参数运行 Pydoc 可查看其全部选项。注意——往日回声 (Blast from the past)：Pydoc 曾经有一个简单的 GUI 模式，由 `-g` 标志调用，会启动一个与服务器通信的 tkinter GUI 客户端。这个模式一直可用到 Python 3.2，当时认为目前的纯浏览器模式已经足够，于是把存在已久的 GUI 模式整个砍掉了。当然，浏览器也是 GUI，而且随浏览器模式出现的 Pydoc CSS 配置方案，假定用户具备许多 Python 用户不具备的额外 Web 开发知识。在开源世界里，那些有时间也有意愿去改变事物的人，为所有人定义了未来。

深度理解

·核心概念（关键警告）：Pydoc 提取文档的机制 = 导入文件。导入 = 执行顶层代码。所以给脚本文件看文档 = 运行一次脚本——这是“Pydoc 会执行你的代码”的真相，也是本章最重要的实用警告之一。

· `if name == 'main':` 的伏笔：把“只有作为脚本运行时才执行”的代码包进这个判断，是 Python 项目的标准防御写法——它的作用正在于：文件被导入时（包括被 Pydoc 导入）顶层只定义名字、不执行动作。这个模式在第五部分（模块）正式学习，但作者在这里就让你知道“为什么需要它”。

- 模块搜索路径与 Pydoc 的可见范围：Pydoc 的索引页 = sys.path 上的内容。理解这一点，就能解释：为什么标准库永远可见（路径包含标准库目录）、为什么自写模块时有时无（取决于启动目录与 PYTHONPATH）。
- -w 保存 HTML 的实用场景：生成静态文档页，脱离 Pydoc 服务器也能分发查看——这是“文档产品化”的第一步，Sphinx 做的正是这件事的工程化版本。
- -g 被砍的历史：tkinter GUI 被浏览器模式取代，是因为浏览器本身就是 GUI、且免维护。这个例子完美诠释了“开源社区如何决定工具命运”：不再维护的功能会被淘汰，替代方案足够好就足以清退旧方案。
- 初学者误区：① 用 Pydoc 查一个“会弹输入框”的脚本，结果控制台被脚本 IO 搞乱——这不是 bug，是机制使然；② 以为 help() 和 Pydoc 对文件内容做静态分析——不，它真的导入并运行模块（对模块而言通常无副作用，因为模块只定义不执行）。

15.8 超越 Docstring: Sphinx (Beyond Docstrings: Sphinx)

英文原文

If you're looking for a way to document your Python programs in a more sophisticated way, you may wish to check out Sphinx—a documentation generator system used to create Python's standard manuals descri

bed in the next section, as well as docs for many other software projects. It uses simple reStructuredText as its markup language and inherits much from the Docutils suite of parsing and translating tools. Among other things, Sphinx supports a variety of output formats, automatic cross-references and indexes, and automatic code highlighting (colorization) using Pygments, which is itself a noteworthy Python tool. This is probably overkill for smaller programs where docstrings and help may suffice, but can yield higher-grade documentation for larger projects. See the web for more details on Sphinx, its related tools, and other options in the docs domain.

中文翻译

如果你在寻找一种更精致的方式来记录你的 Python 程序，可以看看 Sphinx——一个文档生成器系统，用于创建下一节描述的 Python 标准手册，以及许多其他软件项目的文档。它用简单的 reStructuredText 作为标记语言，并从 Docutils 这套解析和翻译工具中继承了相当多东西。除此之外，Sphinx 支持多种输出格式、自动交叉引用（cross-references）和索引，以及用 Pygments 做自动代码高亮（着色）——Pygments 本身就是个值得注意的 Python 工具。对 docstring 和 help 可能就够用的小程序来说，Sphinx 大概是杀鸡用牛刀；但对大型项目，它能产出更高档次的文档。Sphinx 及其相关工具、以及文档领域的其他选择，更多细节请上网搜索。

深度理解

·核心概念：Sphinx 是文档"工程化"工具——docstring 是"贴纸"，Sphinx 是"出版社"：从 docstring + 手写 reStructuredText 页面出发，自动生成带目录、交叉引用、索引、高亮代码的完整文档站点（默认 HTML，也可输出 PDF、ePub 等）。

·它的血缘：Sphinx 构建在 Docutils（解析 reStructuredText 的工具套件）之上；Pygments（高亮库）负责代码着色。这三个都是 Python 生态的文档基础设施——python.org 的官方文档就是 Sphinx 生成的（下一节会讲到）。

·为什么选 reStructuredText 而不是 Markdown 或 HTML：reStructuredText 比 HTML 简单得多（人写易读），比 Markdown 更严谨（有确定的语义结构，能支持交叉引用、索引这类"结构化文档"需求）。这正是 15.5.2 节"手写 HTML 行不通"讨论的答案——社区最终选择了轻量标记语言。

·何时该用：判断标准是项目规模——小型脚本用 docstring + help() 足够；发布给他人使用的库/框架、需要版本化文档站点的项目，才值得引入 Sphinx（现代备选还有 MkDocs + Material、Sphinx 的 Markdown 方言 MyST）。

·生活类比：docstring 是日记，Sphinx 是出版成书——日记随手写，出版需要编辑、目录、索引、装帧，但读者体验完全不同。

·初学者误区：过早学 Sphinx——它需要额外学习 reStructuredText 语法，对还在学语言的新手是负担。作者的排序（先 docstring，后 help，最后才提 Sphinx）本身就是推荐的学习顺序。

15.9 标准手册 (The Standard Manuals)

英文原文

As you know by now, this book is a tutorial that teaches by example. While its index and table of contents can be used to hunt for random topics after the fact, it's mainly designed to be read, not to serve as a reference resource. Given that, you'll probably want to supplement this book with reference tools once you move on to real projects. Among these, Python's standard manuals may provide the most complete and up-to-date reference to the language and its toolset. You can easily view these manuals online with the Documentation link at Python's website. They can also be viewed via the "Manuals" entry in Python's entry in the Start menu on Windows, and can be opened from "Python Docs" in the Help menu within the IDLE coding GUI. See your toolset for other access options. When first opened, the manuals display a root page with a search box, like that captured in Figure 15-3. The two most important entries here are most likely the "Library reference" (which documents built-in types, fu

nctions, exceptions, and standard-library modules) and the "Language reference" (which provides a formal description of language-level details). Both will probably be regular companions once you start coding Python in earnest. Also of notable interest, the "What's new" documents in this standard manual set chronicle Python changes made in each release beginning with Python 2.0, which came out in late 2000—useful for those porting older Python code, or older Python skills. These documents are also useful for uncovering additional details on the differences in the Python 2.X and 3.X language lines, as well as recent Python 3.X changes covered in this book. For both better and worse, change has been a constant in Python since its 0.X days. While the future is impossible to predict, the standard manuals' "What's new" docs will also cover Python mods almost certain to arise after this book's release. It's nearly required reading for anyone working downstream of a perpetually morphing sandbox. (Figure 15-3: Python's standard manuals, available on the web and elsewhere)

中文翻译

你现在应该已经知道，本书是一本以示例教学的教程。虽然它的索引和目录可以用来事后检索随机主题，但它主要是设计来“读”的，而不是充当参考资源。既然如此，一旦你转战真实项目，大概会用参考工具来补充本书。在这些工具中，Python 的标准手册 (standard manuals) 或许提供了关

于这门语言及其工具集最完整、最新鲜的参考。你可以在 Python 官网用 Documentation 链接轻松在线阅读这些手册；在 Windows 上，也可以通过开始菜单 Python 条目里的 "Manuals" 项查看；还能从 IDLE 编码 GUI 的 Help 菜单里打开 "Python Docs"。其他访问方式请查看你的工具集。首次打开时，手册显示一个带搜索框的根页面，即图 15-3 所示。这里最重要的两个条目很可能是 "Library reference"（库参考，记录内置类型、函数、异常和标准库模块）和 "Language reference"（语言参考，提供语言级细节的正式描述）。一旦你开始认真用 Python 编码，这两者大概会成为你的常伴。另一个值得注意的是，这套标准手册中的 "What's new"（更新说明）文档，从 2000 年底发布的 Python 2.0 起，逐年记录了每个版本的 Python 变更——对移植旧 Python 代码、或旧 Python 技能的人来说非常有用。这些文档还有助于挖掘 Python 2.X 和 3.X 语言线差异的更多细节，以及本书所涉及的近期 Python 3.X 变更。无论好坏，自 0.X 时代以来，变化一直是 Python 的常态。虽然未来无法预测，但标准手册的 "What's new" 文档也会涵盖本书出版之后几乎必然出现的 Python 改动。对任何工作在 "永远在变形的沙盒" 下游的人来说，这几乎算是必读。

（图 15-3：Python 的标准手册，可在网上及其他渠道获取）

深度理解

·核心概念：教程（本书）与参考手册（官方文档）是两种不同的文本——教程按学习曲线组织，手册按 API 组织结构。它们的分工是：教程教你 "怎么想"，手册告诉你 "确切的细节"。程序员最后都要以手册为最终裁判（因为手册描

述的是解释器的真实行为)。

·两大手册的分工：

· Library Reference (库参考)：查"有什么工具、怎么调"——内置类型、函数、异常、标准库模块，日常 80% 的查询都在这；

· Language Reference (语言参考)：查"语言本身的确切语义"——词法规则、数据模型、执行模型，是"正式规范"，内容最深、最技术化。

· "What's new" 的价值：每个版本的变更日志是"语言演化的编年史"。对老代码、老技能的人尤其重要——例如 Python 2 的 `print` 语句到 3 的 `print()` 函数、`xrange`→`range` 等破坏性变更都在这些文档里有完整记录。这也是紧跟 Python 版本演进 (3.12、3.13...) 的唯一可靠途径。

。

· "永远在变形的沙盒"：作者用沙盒比喻 Python 生态——地基在移动，手册是唯一的地图。这提醒你：读书时要带着"以官方手册为准"的意识，任何书（包括本书）都可能滞后于最新版本。

·生活类比：本书像驾校教练教的驾驶技巧，官方手册像交通法规原文——教练让你开得顺，法规让你知道边界；出了争议，看法规（手册）。

·初学者误区：① 遇到问题只上网搜博客，不查官方手册——博客可能过时或错误，手册是权威；② 觉得官方文档"太难读"就放弃——Library Reference 按模块组织，配合搜索框其实很好用；③ 分不清 Tutorial 与 Reference

——官网首页的 Tutorial 是给初学者的，Library Reference 才是"字典"。

15.10 网络资源 (Web Resources)

英文原文

Finally, besides the standard manuals, Python's website also hosts additional resources, some of which cover special topics or domains, including non-English Python resources and introductions scaled to different target audiences. Today you will also find a multitude of Python blogs, websites, wikis, and other resources on the web at large. Given the explosion of the web in recent decades, some online resources are naturally more authoritative and reliable than others, and some are sadly geared more toward monetization than education. This word of caution includes AI chatbots, which can only do as well as the data they're fed and paraphrase, and are not a replacement for the deep learning needed to work in software. That said, there's a wealth of Python material out there to be had—for those willing to exercise the prudence and skepticism that today's web demands.

中文翻译

最后，除了标准手册，Python 官网还承载着其他资源，其中一些覆盖特殊主题或领域，包括非英语的 Python 资源，

以及面向不同目标受众的入门介绍。如今，你在整个互联网上还会发现大量的 Python 博客、网站、维基和其他资源。鉴于近几十年来互联网的爆炸式发展，有些在线资源自然比另一些更权威、更可靠；而有些可悲地更侧重于变现而不是教育。这条提醒也包括 AI 聊天机器人——它们只能做到“被投喂的数据”以及转述的水平，并不能替代在软件行业工作所需的深度学习。话虽如此，外面依然有海量的 Python 素材可挖——只要你有意愿，能够行使当今网络所要求的谨慎和怀疑精神。

深度理解

·核心概念：官方资源之外还有“非官方海洋”——质量参差。作者给的核心原则就两个字：审慎（prudence + skepticism）。网络资源是放大器：好的内容放大学习速度，坏的内容放大困惑。

·信息源分级（现代实践的判断框架）：① 官方（python.org 文档、PEP、官方教程）——最高权威；② 高信誉第三方（Real Python、Python 官方博客、知名图书作者）——通常准确；③ 普通博客/论坛（Stack Overflow 等）——需要交叉验证；④ 营销型内容与 AI 生成内容——需警惕。判断标准：内容是否引用官方文档、是否给出可复现的代码、是否标注版本。

·关于 AI 聊天机器人的提醒：书里这段写于 AI 普及初期，但今天更有现实意义——LLM 是“转述引擎”不是“验证引擎”，可能流畅地给出过时或错误的 API 用法。作者主张：AI 可以当辅导老师，但最终要以手册和实际运行结果为准——“深度学习”在此一语双关：既指语言学习，也指

知识内化。

·初学者误区：① 收藏夹堆满教程却从不查手册——资源多不等于知识多；② 轻信没有版本标注的博客代码——Python 3.12 里合法的东西在旧版本可能报错（如 \ 转义弃用）；③ 用 AI 生成代码却不运行验证——不跑的代码等于没写。

·生活类比：网络资源像自助餐厅——菜品丰富，但新鲜程度不一；官方手册是“菜单上的食材产地”，AI 是“帮你端菜的机器人”——它不知道菜新不新鲜。

15.11 常见编码陷阱 (Common Coding Gotchas)

英文原文

Before the programming exercises for this part of the book, let's run through some of the most common mistakes beginners make when coding Python statements and programs. Many of these are warnings issued earlier in this part of the book, collected here for ease of reference. You'll learn to avoid these pitfalls once you've gained a bit of Python coding experience, but a few words now might help you avoid falling into some of these traps initially: Don't forget the colons. Always remember to type a `:` at the end of compound statement headers—the first line of an `if`, `match`, `while`, `for`, etc. You'll probably forget at first (as have thou

sands of Python students over the years), but you can take some comfort from the fact that it will soon become an unconscious habit. Start in column 1. Be sure to start top-level (unnested) code in column 1. That includes unnested code typed into module files, as well as unnested code typed at the interactive prompt (a.k.a. REPL). Indented means nested in Python, so it doesn't work at the top. Blank lines matter at the interactive prompt. Blank lines in compound statements are always irrelevant and ignored in module files, but when you're typing code at the interactive prompt, they end the statement. In other words, blank lines tell the interactive REPL that you've finished a compound statement; if you want to continue, don't hit the Enter key at the ... prompt (if shown) until you're really done. This also means you can't paste multiline code at this prompt; it must run one full statement at a time, with blank lines as needed. Indent consistently. Avoid mixing tabs and spaces in the indentation of a nested block. Otherwise, what you see in your editor may not be what Python sees when it counts tabs as a number of spaces. This is true in any block-structured language, not just Python—if the next programmer has tabs set differently, it will be difficult or impossible to understand the structure of your code. It's safer to use all tabs or all spaces for each block. Don't code C in Python. A reminder for C/C++ programmers: you don't need to type parentheses around tests in if and

while headers (e.g., if (X==1)). You can if you like (any expression can be enclosed in parentheses), but they are fully superfluous in this context. Also, do not terminate all your statements with semicolons; it's technically legal to do this in Python as well, but it's totally useless unless you're placing more than one statement on a single line (the end of a line normally terminates a statement). And remember, don't use {} around blocks (indent your nested code blocks consistently instead), and avoid embedding assignment statements in while loop tests unless they're simple (even though Python now enables this with :=). Use simple for loops instead of while or range. Another reminder: a simple for loop (e.g., for x in y) is almost always simpler to code and often quicker to run than a while- or range-based counter loop. Avoid the temptation to count things in Python; though occasionally required, it's usually subpar. Beware of mutables in assignments. As cautioned before, be careful about using mutables in a multiple-target assignment (a = b = []), as well as in an augmented assignment (a += [1, 2]). In both cases, in-place changes may impact other variables. See Chapter 11 for details if you've forgotten why this is true. Don't expect results from functions that change objects in place. We encountered this nag earlier, too: in-place change operations like the list.append and list.sort methods presented in Chapter 8 do not return values (other than None), so you should call the

m without assigning the result. It's not uncommon for beginners to say something like `mylist = mylist.append(X)` to try to get the result of an append, but this assigns `mylist` to `None`, not to the modified list (in fact, you'll lose your reference to the list altogether). Always use parentheses to call a function. You must add parentheses after a function name to call it, whether it takes arguments or not (e.g., use `function()`, not `function`). In the next part of this book, you'll learn that functions are simply objects that have a special operation—a call that you trigger with the parentheses—but they can be referenced like any other object without triggering a call. This often crops up with files; it's common for beginners to type `file.close` to close a file, rather than `file.close()`. Because it's legal to reference a function without calling it, the first version succeeds silently, but it does not close the file. Don't use extensions or paths in imports and reloads. Omit directory paths and file extensions in import statements—say `import mod`, not `import mod.py`. We discussed module basics in Chapter 3 and will continue studying modules in Part V. Because modules may have other extensions besides `.py` (e.g., `.pyc` for bytecode), hard coding a particular extension is not only invalid syntax; it doesn't make sense. Python picks an extension automatically, and any platform-specific directory path syntax comes from module search path settings, not the import statement. Until we explore this in more

depth, use just a simple name in imports. And other pitfalls in other parts. Be sure to also see the built-in type warnings at the end of Part II, as they may qualify as coding issues too. There are additional "gotchas" that crop up commonly in Python coding—losing a built-in function by reassigning its name, hiding a library module by using its name for one of your own, changing mutable argument defaults, and so on—but we don't have enough background to cover them yet. To learn more about both what you should and shouldn't do in Python, you'll have to read on; later parts extend the set of "gotchas" and fixes we've enumerated here.

中文翻译

在本书这一部分的编程练习之前，让我们过一遍初学者编写 Python 语句和程序时最常犯的一些错误。其中许多是本部分前面章节已经发出的警告，为了便于查阅，这里集中在一起。一旦你有了一些 Python 编码经验，你就会学会避开这些坑，但现在说几句，也许能帮你一开始就不掉进这些陷阱：别忘了冒号。永远记得在复合语句头（compound statement headers）末尾输入：——即 if、match、while、for 等语句的第一行。你一开始大概会忘（多年来成千上万的 Python 学生都是如此），但可以宽慰的是，它很快会变成无意识习惯。从第 1 列开始。一定要把顶层（不嵌套的）代码放在第 1 列开始。这包括输入模块文件的非嵌套代码，也包括在交互提示符（即 REPL）下输入的非嵌套代码。缩进在 Python 里意味着嵌套，所以顶层不能用缩进。交互

提示符下空行很重要。复合语句中的空行在模块文件中总是无关紧要、会被忽略的；但在交互提示符下输入代码时，空行会结束语句。换句话说，空行告诉交互式 REPL“你已经写完一条复合语句了”；如果想继续，在真正写完之前别在 ... 提示符（如果显示）处按回车。这也意味着你不能在这个提示符下粘贴多行代码；它必须一次运行一整条语句，按需留空行。缩进要一致。避免在嵌套块的缩进中混用制表符（Tab）和空格。否则，当 Python 把 Tab 按一定空格数计算时，你在编辑器里看到的可能与 Python 看到的不是一回事。这适用于任何块结构语言，不限于 Python——如果下一位程序员 Tab 设置不同，理解你的代码结构将困难甚至不可能。对每个块，全用 Tab 或全用空格更安全。别用 C 的风格写 Python。给 C/C++ 程序员的提醒：if 和 while 头的测试条件不需要加括号（如 if (X==1)）。你想加也行（任何表达式都可以括在括号里），但在此语境下完全是多余的。同样，不要用分号结束所有语句；Python 里这样做技术上合法，但完全没用——除非你要在一行里放多条语句（行尾通常就是语句的终止）。还有，记住别用 {} 包代码块（要一致地缩进你的嵌套代码块），并且除非很简单，否则避免在 while 循环测试里嵌赋值语句（虽然 Python 现在用 := 支持了这一点）。用简单的 for 循环而不是 while 或 range。再提醒一次：简单的 for 循环（如 for x in y）几乎总是比基于 while 或 range 的计数器循环更容易编写，而且往往运行更快。在 Python 里要抵制“数数”的诱惑；虽然偶尔必需，但通常都是次优方案。当心赋值中的可变对象（mutables）。正如之前警告过的，在多目标赋值（a = b = []）以及增强赋值（a += [1, 2]）中使用可变对象时要小心。两种情况下的原地修改都可能影响其他变量。如果忘了

为什么，参见第 11 章的细节。别指望原地修改对象的函数有返回值。我们之前也遇到过这个老问题：`list.append` 和 `list.sort` 这类原地修改操作（第 8 章介绍）不返回值（除了 `None`），所以调用它们时不应把结果赋给变量。新手常说 `mylist = mylist.append(X)` 想拿到 `append` 的结果，但这会把 `mylist` 赋成 `None`，而不是修改后的列表（事实上你会彻底丢掉对这个列表的引用）。调用函数一定要用括号。函数名后面必须加括号才能调用它，无论它有没有参数（例如用 `function()`，而不是 `function`）。在本书下一部分你会学到：函数只是带有一种特殊操作（用括号触发的调用）的对象——但你可以像引用任何其他对象一样引用函数而不触发调用。这在文件操作上经常出问题；初学者常用 `file.close` 而不是 `file.close()` 来关闭文件。因为“引用而不调用”是合法的，第一种写法会静默成功，但它并没有关闭文件。`import` 和 `reload` 不要带扩展名或路径。在 `import` 语句里省略目录路径和文件扩展名——写 `import mod`，而不是 `import mod.py`。我们在第 3 章讨论过模块基础，第五部分将继续研究模块。因为模块可能有 `.py` 之外的其他扩展名（如字节码的 `.pyc`），硬编码特定扩展名不只是语法无效，更是不合逻辑。扩展名由 Python 自动挑选，任何平台相关的目录路径语法都来自模块搜索路径设置，而不是 `import` 语句。在我们深入探索之前，`import` 里就用一个简单的名字。其他部分的更多坑。也别忘了看第二部分末尾的内置类型警告，它们可能也够格算编码问题。Python 编码中还有一批常见的“坑”——把内置函数的名字重新赋值而弄丢它、用自己的模块名遮蔽库模块、修改可变的参数默认值等等——但我们目前的背景知识还不够覆盖它们。要了解更多 Python 中该做什么、不该做什么，你得继续读下去；

后面的部分会扩展我们在这里列出的"坑"和修复方法的清单

。

深度理解

·核心概念：这十一条是第三部分全部"前车之鉴"的集中营——每条都对应一个真实报错或静默 bug。它们的共性：Python 的语法规则大多来自"缩进即结构"这一个核心决策（冒号引入块、缩进表示嵌套、行尾结束语句），理解了这一条，一半陷阱自动消失。

·为什么空行在 REPL 和文件里行为不同：模块文件由编译器一次解析整个文件，空行自然无关紧要；REPL 是"逐条语句解析"的——空行是解释器判断"复合语句是否输入完毕"的哨兵信号。这是交互式解释执行的直接后果，也是"粘贴多行代码失败"的根源。

·Tab 与空格混用的本质：Python 把 Tab 当做一个逻辑缩进单位（对 8 个空格），与编辑器显示宽度可能不同——"所见非所得"。这就是 PEP 8 推荐 4 空格且禁 Tab 混用的原因。

·"别用 C 风格写 Python"的深层原因：括号、分号、花括号在 C 里是语法必需，在 Python 里是纯噪音——因为 Python 的设计目标是"可读代码优先"，语法元素越少，代码越干净。:= 海象运算符（第 11 章）虽支持在条件里赋值，但"简单才用"是社区共识。

·可变对象陷阱回顾：a = b = [] 让两个名字指向同一个列表对象，任何 a.append() 都会影响 b；a += [1,2] 对列表是原地扩展，同样会影响其他引用者。根因是"名字是引

用、可变对象原地变化"的内存模型（第 6、11 章）。

- `mylist = mylist.append(X)` 的经典事故：`append` 返回 `None`——把 `mylist` 覆盖为 `None` 就再也找不到原列表。这个 bug 新手几乎必踩一次，根因是把"原地修改"方法当成"返回新值"的函数。对比：`sorted()` 返回新列表，`list.sort()` 原地修改——记住"原地修改的方法返回 `None`"这个规律。

- `file.close` 不调用 bug：函数是对象，引用它合法且静默——错误不会报，文件却没关。这也是"函数调用必须带括号"为什么是 Python 新手第一条铁律。

- 初学者误区总结：① 以为这些"坑"会随经验自然消失——不会，它们会换个样子出现；② 只记报错信息不记根因——报错（如 `SyntaxError`）反而好修，静默错误（`None` 赋值、`file.close`）才致命；③ 把"能运行"当"正确"——许多陷阱正是"运行了但行为不对"。

15.12 章节总结 (Chapter Summary)

英文原文

This chapter toured documentation—both documentation we write ourselves in our own programs, and documentation available for tools we use. We met doc strings, explored reference resources for Python, and learned how `Pydoc`'s help function and browser-based interfaces provide extra sources of documentation. Because this is the last chapter in this part of the boo

k, we also reviewed common coding mistakes to help you avoid them. In the next part of this book, we'll start applying what we already know to larger program constructs. Specifically, the next part takes up the topic of functions—a tool used to group statements for reuse. Before moving on, however, be sure to work through the set of lab exercises for this part of the book that appear at the end of this chapter. And even before that, let's run through this chapter's quiz to review

中文翻译

本章巡游了文档——既包括我们自己在程序中编写的文档，也包括我们所用工具的文档。我们认识了 docstring，探索了 Python 的参考资源，学习了 Pydoc 的 help 函数和基于浏览器的界面如何提供额外的文档来源。因为这是本书本部分的最后一章，我们还回顾了常见编码错误，帮你避开它们。在本书的下一部分，我们将开始把已学知识应用到更大的程序构造上。具体地说，下一部分将展开函数 (functions) 这个主题——一种把语句分组以便复用的工具。不过在继续之前，请务必完成本章末尾的本部分实验练习。而在这之前，让我们先过一遍本章测验来复习。

深度理解

·承上启下：本章的定位是“第三部分的句号”——文档是“工程的软实力”，练习是“语法的总验收”。下一部分（第四部分）进入函数，是全书“从语句到组件”的关键转折：语句只能线性执行，函数让代码有了复用和抽象。

·为什么要先做练习再学函数：第三部分练习覆盖 for/if/match/while、推导式、字典排序等全部语句知识——作者在章节结构上刻意安排“旧知识巩固 → 新概念展开”，符合“间隔复习”的学习科学。

·本章知识在下一部分的使用：dir()、help()、docstring 在学函数时马上就是工具——写完函数 help() 它，签名和 docstring 一目了然。文档能力不是孤立的，它是学习新内容时的加速器。

15.13 自测：Quiz（随章测验）

测验题目（中英对照）

1. When should you use documentation strings instead of hash-mark comments? 什么时候应该用文档字符串 (docstring) 而不是井号 (#) 注释? 2. Name three ways you can view documentation strings. 说出三种查看文档字符串的方式。3. How can you obtain a list of the available attributes in an object? 如何获得一个对象中可用属性的列表? 4. How can you get a list of all importable modules on your computer? 如何获得你计算机上所有可导入模块的列表?

参考答案

1. Documentation strings (docstrings) are considered best for larger, functional documentation, describing the use of modules, functions, classes, and methods in your code. Hash-mark comments are better limited

to smaller-scale documentation about arcane expressions or statements at strategic points on your code. This is partly because docstrings are easier to find in a source file (they have specific locations), but also because they can be extracted and displayed by the Pydoc system.文档字符串 (docstring) 被认为最适合较大规模的功能性文档, 描述你代码中模块、函数、类和方法的使用。井号注释更适合局限于较小规模的文档——针对代码中战略位置的晦涩表达式或语句。这既是因为 docstring 在源文件中更容易找到 (它们有特定位置), 也是因为它们能被 Pydoc 系统提取和显示。

2. You can see docstrings by printing an object's doc attribute, by passing it to Pydoc's help function, and by selecting modules in Pydoc's HTML-based browser interface. The latter is launched with a -b command-line switch, and runs a client/server system that displays documentation in a popped-up web browser. Pydoc can also be run to save a module's documentation in an HTML file for later viewing or printing.查看 docstring 的方式: 打印对象的 doc 属性; 把它传给 Pydoc 的 help 函数; 以及在 Pydoc 的 HTML 浏览器界面中选择模块。后者用 -b 命令行开关启动, 运行一个客户端/服务器系统, 在弹出的 Web 浏览器中显示文档。Pydoc 也可以运行来把模块文档保存为 HTML 文件, 供日后查看或打印。

3. The built-in dir(X) function returns a list of all the attributes attached to any object. A list comprehension like [a for a in dir(X) if not a.startswith('_')] can be used to filter out internals' names with underscores (you'll learn how to wrap thi

s in a function in the next part of the book to make it reusable).内置 `dir(X)` 函数返回附着在任何对象上的所有属性的列表。像 `[a for a in dir(X) if not a.startswith('_)]` 这样的列表推导式可以用来过滤掉带下划线的内部名字（本书下一部分你会学到如何把它包进函数以复用）。4. Because the index page opened for Pydoc's browser-based -b mode displays every module on your import search path, this shows all the modules available for imports in your programs. This relies on configurable search-path settings that we'll cover more fully later in this book, but it includes both the current directory and Python's standard library by default. You can also write code of your own to achieve this using the same tools the Pydoc uses (it's just Python code, after all), but it's a significant manual task.因为 Pydoc 基于浏览器的 -b 模式打开的索引页会显示你导入搜索路径上的每个模块，所以它展示了你的程序中所有可导入的模块。这依赖于可配置的搜索路径设置——本书后面会更完整地讲解——但默认情况下它既包含当前目录，也包含 Python 的标准库。你也可以用 Pydoc 所用的同样工具自己写代码实现这一点（毕竟那只是 Python 代码），但那是个不小的人工活儿。

深度理解

这四道题是本章的“考点地图”：① 文档分工（docstring vs #）；② 三种查看方式（doc、help、浏览器）；③ `dir()` + 推导式过滤；④ Pydoc 索引页与模块搜索路径的关系。全部覆盖了“写文档”和“查文档”两条主线。

·第三题的答案技巧：`dir()` 给的列表很"脏"（含大量 X 内部名字），过滤写法本身（`startswith("")`）就是一个迷你 API——后续章节会把它包成可复用函数，这正是"工具意识"的雏形：见到重复劳动，就想封装。

·第四题的答案提醒：你不需要自己写代码遍历 `sys.path`——Pydoc 索引页已经替你做了。善用工具也是一种能力；而答案最后那句"你也可以自己写"则点出：理解工具的原理（Pydoc 只是 Python 代码），你就有能力复制它。

15.14 第三部分练习 (Test Your Knowledge: Part III Exercises)

英文原文

Now that you know how to code basic program logic, the following exercises will ask you to implement some simple tasks with statements. Much of the work is in exercise 4, which lets you explore coding alternatives. There are always many ways to arrange statements, and part of learning Python is learning which arrangements work better than others. You'll eventually gravitate naturally toward what experienced Python programmers call "best practice," but best practice takes practice. See "Part III, Statements and Syntax" in Appendix B for solutions to the following exercises:

中文翻译

现在你已经知道如何编写基本的程序逻辑，以下练习要求你用语句实现一些简单任务。大部分工作在练习 4，它让你探索不同的编码方案。语句的排列方式总是有很多种，学习 Python 的一部分就是学习哪种排列优于其他。你最终会自然而然地趋向于经验丰富的 Python 程序员所说的"最佳实践" (best practice)，但最佳实践需要实践。以下练习的解答见附录 B 的"第三部分：语句与语法"。

15.14.1 练习 1：编写基本循环 (Coding basic loops)

英文原文

This exercise asks you to experiment with for loops. a. Write a for loop that prints the ASCII code point of each character in a string named S. Use the built-in function `ord(character)` to convert each character to its integer code point. This function technically returns a Unicode code point that may not fall in ASCII's range, but if you restrict its content to ASCII characters, you'll get back ASCII codes. (Test it interactively to see how it works, if needed.) b. Next, change your loop to compute the sum of the ASCII code points of all the characters in a string. c. Finally, modify your loop again to return a new list that contains the ASCII code points of each character in the string. Does the expression `map(ord, S)` have a similar effect? How about `[o`

rd(c) for c in S]? Why? (Hint: see Chapter 14.)

中文翻译

本练习要求你实验 for 循环。a. 写一个 for 循环，打印字符串 S 中每个字符的 ASCII 码点。用内置函数 ord(字符) 把每个字符转换成它的整数码点。从技术上讲这个函数返回的是 Unicode 码点，可能不在 ASCII 范围内，但如果你把内容限定为 ASCII 字符，拿回来的就是 ASCII 码。（如果需要，交互式测试一下看它是怎么工作的。）b. 接下来，修改你的循环，计算字符串中所有字符的 ASCII 码点之和 (sum)。c. 最后，再修改循环，返回一个包含字符串中每个字符 ASCII 码点的新列表。表达式 map(ord, S) 有类似效果吗？[ord(c) for c in S] 呢？为什么？（提示：见第 14 章。）

15.14.2 练习 2：编写基本选择 (Coding basic selections)

英文原文

Write Python if and match statements that print the first three month names of the year, given their relative numbers. For example, given 1, the output should be January, and for 3, it should be March (or whatever months are named in your locale). Then do the same by coding the choice with both a dictionary-key index and a list-offset index. How would you handle out-of-range month numbers?

中文翻译

编写 Python 的 if 和 match 语句：给定月份的相对编号，打印一年中前三个月份的名字。例如，给 1 输出 January，给 3 输出 March（或者你本地化环境里月份的名字）。然后用字典键索引和列表偏移索引两种方式实现同样的选择逻辑。你会怎么处理超出范围的月份编号？

15.14.3 练习 3：反斜杠字符 (Backslash characters)

英文原文

What happens on your machine when you type the following code interactively? `python for i in range(50): print(f'hello {i}\a')` Beware that if it's run outside of some interfaces like IDLE, this example may beep at you, so you may not want to run it in a crowded room! IDLE ignores odd characters instead of beeping—spoiling much of the joke (see the backslash escape characters in Table 7-2).

中文翻译

在交互模式下输入以下代码，你的机器上会发生什么？`python for i in range(50): print(f'hello {i}\a')` 小心：如果在 IDLE 之外的某些界面运行，这个示例可能会对你“哔哔”叫，所以你可能不想在拥挤的房间里运行它！IDLE 会忽略奇怪字符而不发出哔声——这破坏了大半的玩笑（见第 7 章表 7-2 的反斜杠转义字符）。

深度理解 (练习 3 详解)

· \a 是什么: \a 是转义序列 "bell" (响铃) ——终端收到这个字符会发出提示音。循环 50 次就哔 50 次。这是第 7 章转义字符表的实战复习: 字符串里的反斜杠转义会被解释成控制字符。

· 为什么 IDLE 不响: IDLE 是文本 GUI, 对 \a 等控制字符直接忽略而非送往终端响铃——同样的代码在不同宿主上行为不同, 这正是"运行环境影响行为"的活教材。

· f-string 的复习: f'hello {i}\a' 中 {i} 被求值替换, \a 是字面转义——f-string 与转义并存, 二者不冲突 (第 14 章)。

15.14.4 练习 4: 字典排序 (Sorting dictionaries)

英文原文

In Chapter 8, we saw that dictionaries are collections that store keys by insertion order only. Write a for loop that prints a dictionary's items in sorted (ascending) key/value order. (Hint: store keys in unordered fashion, and use the dictionary keys and list sort methods, or the sorted built-in function.)

中文翻译

在第 8 章我们见过: 字典是以插入顺序存储键的集合。写一个 for 循环, 按键/值的升序打印字典的条目。(提示:

以无序方式存储键，用字典的 `keys` 方法和列表的 `sort` 方法，或内置 `sorted` 函数。)

深度理解 (练习 4 详解)

·关键背景：Python 3.7+ 字典保持插入顺序（这是 CPython 3.6 的实现细节升级为语言保证）——但“插入顺序”≠“排序顺序”。要按升序打印，必须显式排序。

·两条路线：① 传统：`keys = list(d.keys()) + keys.sort()` 再遍历；② 现代：`for k in sorted(d): print(k, d[k])`——`sorted()` 返回新排序列表，不修改原字典。第二条更 Pythonic (`sorted` 是 15.11 节“用简单工具”精神的体现)。

·这个练习的深意：把“数据”与“呈现顺序”分离——排序是视图层面的需求，不该改动存储结构。这也是后面学函数/类时“职责分离”思想的雏形。

15.14.5 练习 5：程序逻辑替代方案 (Program logic alternatives)

英文原文

Consider the following code, which uses a while loop and found flag to search a list of powers of 2 for the value of 2 raised to the fifth power (32). It's stored in a module file called `power.py`:

```
python L = [1, 2, 4, 8, 16, 32, 64]
X = 5
found = False
i = 0
while not found and i < len(L):
    if 2 ** X == L[i]:
        found = True
    else:
        i = i + 1
if found:
    print('at index', i)
else:
    print(X, 'not found')
```


\$ python3 power.py
at index 5
As is, the example does

n't follow normal Python coding techniques. Follow the steps outlined here to improve it (for all the transformations, you may either type your code interactively or store it in a script file run from the system command line or other interface—using a file makes this exercise much easier):

- First, rewrite this code with a while loop else clause to eliminate the found flag and final if statement.
- Next, rewrite the example to use a for loop with an else clause, to eliminate the explicit list-indexing logic. (Hint: to get the index of an item, use the list index method—`L.index(X)` returns the offset of the first X in list L.)
- Next, remove the loop completely by rewriting the example with a simple in operator membership expression. (See Chapter 8 for more details, or type this to test: `2 in [1, 2, 3]`.)
- Finally, use a for loop and the list append method to generate the powers-of-2 list (L) instead of hardcoding a list literal.

Deeper thoughts:

- Do you think it would improve performance to move the `2 * x` expression outside the loops? How would you code that?
- As we saw in exercise 1, Python includes a `map(function, iterable)` tool that can generate a powers-of-2 list, too: `map(lambda x: 2 * x, range(7))`. Try typing this code interactively; you'll meet lambda more formally in the next part of this book, especially in Chapter 19. Would a list comprehension help here (see Chapter 14)? How about a `:=` expression (see Chapter 11)?

中文翻译

考虑下面的代码，它用 while 循环和 found 标志在一个 2 的幂列表中查找 2 的 5 次方 (32)。它存储在一个名为 power.py 的模块文件中：

```
python L = [1, 2, 4, 8, 16, 32, 64]
X = 5
found = False
i = 0
while not found and i < len(L):
    if 2 * X == L[i]:
        found = True
    else:
        i = i + 1
if found:
    print('at index', i)
else:
    print(X, 'not found')
```

\$ python3 power.py at index 5

就目前而言，这个示例不符合正常的 Python 编码技巧。请按下面的步骤改进它（所有转换，你既可以交互式输入代码，也可以存入脚本文件从系统命令行或其他界面运行——用文件会让这个练习容易得多）：

- a. 首先，用 while 循环的 else 子句重写这段代码，去掉 found 标志和最后的 if 语句。
- b. 接下来，用带 else 子句的 for 循环重写示例，去掉显式的列表索引逻辑。（提示：要拿到某个条目的索引，用列表的 index 方法——L.index(X) 返回列表中第一个 X 的偏移量。）
- c. 接下来，完全去掉循环，用简单的 in 运算符成员表达式重写示例。（更多细节见第 8 章，或输入这个测试：2 in [1, 2, 3]。）
- d. 最后，用 for 循环和列表 append 方法生成 2 的幂列表 (L)，而不是硬编码列表字面量。深入思考：

- a. 你认为把 2 * X 表达式移到循环外面能改善性能吗？你会怎么编写？
- b. 如练习 1 所见，Python 有一个 map(函数, 可迭代对象) 工具也能生成 2 的幂列表：map(lambda x: 2 * x, range(7))。试试交互式输入这段代码；你会在本书下一部分、尤其是第 19 章更正式地遇到 lambda。这里用列表推导式（见第 14 章）有帮助吗？:= 表达式（见第 11 章）呢？

深度理解 (练习 5 详解)

· 这个练习的主题：渐进式简化 (refactoring)。五步改造的本质是同一逻辑的五个表达层次：手工计数器 (while+flag) → 循环语义 (while-else) → 迭代 (for-else) → 成员测试 (in) → 数据生成 (append 循环 / map / 推导式)。每一步都在把"实现细节"换成"意图表达"——这是 Pythonic 编程的核心训练。

· while...else / for...else 的语义：循环正常结束 (未被 break 打断) 时执行 else 块。用它做"查找成功/失败"的分支，天然替代 flag——这是许多语言没有的语法，也是新手最容易忽略的 Python 特性。

· in 运算符一行解决：32 in L 由解释器调用列表的 contains 完成查找——C 层实现，快且干净。查找类需求优先想 in 而不是自己写循环。

· 性能问题的标准答案：2 * x 不依赖循环变量 i，是循环不变式 (loop-invariant) ——应在循环前算出一次：p = 2 * x。解释器虽然不会自动提升它 (Python 无此优化，每次迭代都重新求值)，但程序员手动提升是常识。

· map(lambda x: 2 * x, range(7))：lambda 是匿名函数 (第 19 章)，map 惰性求值 (第 14 章) ——返回迭代器而非列表，需 list() 包一层才见结果。列表推导式 [2 * x for x in range(7)] 更直白——这正是"推导式常优于 map+lambda"的著名论战主题。

· 初学者误区：① 写 while 循环时忘了 i = i + 1 导致死循环；② 不知道 else 子句存在，全靠 flag；③ 以为"循环

越多代码越正规"——恰恰相反，Python 鼓励用更高级的抽象（in、推导式、map）消灭循环。

本章总结

技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

- docstring + doc: 团队代码库的"组件说明书"。现代 Python 库（pandas、requests、Django）的每个公开函数都带 docstring——它同时服务人类阅读（help()）和机器提取（Sphinx 生成 API 文档）。
- help() 的日常地位: 任何 REPL/IPython/Jupyter 环境里，"不会用的函数先 help() 一下"是职业习惯。help() 里没有的细节再查官方 Library Reference——这是"自学闭环"的标准路径。
- dir() 的探测价值: 接手陌生对象（尤其第三方库实例）时，dir(obj) + 推导式过滤是"逆向侦察"的第一步——先看有哪些属性，再逐一对感兴趣的属性 help()。
- Pydoc 与团队内部文档: python3 -m pydoc -w 你的模块可以把模块文档导出成 HTML 文件，直接挂到内网文档站——不引入 Sphinx 也能享受"文档产品化"。
- doctest 的相关性: 本书本章未展开，但值得知道——标准库 doctest 模块会运行 docstring 里的示例代码并比对输出，把文档变成可执行的测试（python -m doctest

-v 文件.py) 。docstring 因此可以一鱼两吃：文档 + 回归测试。这是 Python 文档生态的独特优点（其他语言少有"注释即测试"）。

2. 与其他语言（Java/C++）的区别

维度	Python	Java	C++
注释类型	# 注释 + docstring（运行时对象属性）	//、// + Javadoc（独立注释格式）	//、//（无官方文档格式）
文档可编程性	doc 是真实属性，运行时可用	Javadoc 注释经工具提取，运行时不可见	注释纯文本，运行时不可见
内置文档工具	help()、Pydoc 随标准库自带	javadoc 命令需 JDK 自带工具	Doxygen 需第三方安装
文档与测试联动	doctest 直接执行 docstring 示	JUnit 需另写测试类	Google Test 等需另写
注释风格	缩进即结构，无块注释语法	块注释丰富	块注释丰富
标准参考	官方手册（Library + Language Reference） 在线权威	JavaDoc + 官方规范（JLS）	cppreference、ISO 标准

核心洞察：Python 是唯一把"文档作为运行时数据"的主流语言——Javadoc 注释在编译器眼里就是垃圾，而 Python 的 docstring 是对象的一部分。这带来连锁反应：help()、Pydoc、Sphinx、doctest 全部建立在这一个设计决定之上。

3. Python 发展历史背景

· docstring 的起源：Python 1.x 时期，Guido 从 Unix 手册页文化和 Lisp 的"属性即数据"思想获得启发，把"字符串放在定义体开头"约定为文档，并存入 doc。这是"约

定优于配置"的早期范例。

- Pydoc 的诞生：1998 年前后由 Ka-Ping Yee 编写，2000 年随 Python 1.6/2.0 进入标准库。它的设计目标就是让文档"像 man 手册一样"触手可得。
- PEP 257 (2001)：正式规范 docstring 约定——三引号、首行摘要句、多行结构。它和 PEP 8 一起成为 Python 代码外观的"宪法"。
- Sphinx (2008, Python 2.6/3.0 时代)：由 Georg Brandl 开发，替代了旧的 LaTeX 文档方案——python.org 官方文档自此用 Sphinx 生成，并迅速成为开源文档的事实标准 (Read the Docs 平台即基于它)。
- help() 与 REPL 文化：Python 的交互式哲学 ("REPL 是第一文档") 由 IDLE 和 pydoc 共同塑造——在任何环境里按 help() 就能读书，这种体验至今是 Python 学习者的特权。
- 3.13 的 docstring 演进：编译器层面剥离 docstring 前导缩进以节省字节码空间——说明"文档基础设施"仍在持续优化，即使功能上早已成熟。

4. 高级开发者需要掌握的相关知识

- inspect 模块：Pydoc 的"侦察兵"。inspect.getdoc (取 doc 并清理缩进)、inspect.signature (取签名)、inspect.getsource (取源码文本) ——写调试工具、测试框架、ORM 的必备。
- doc 与元编程：装饰器 (第 19 章) 和类装饰器 (第 32 章) 容易"吞掉"被包装对象的 doc, 工程上常用 functools

s.wraps 修复——这是"文档作为运行时数据"在实际工程中的著名坑点。

·文档生成工程：Sphinx + autodoc (从 docstring 自动提取)、MkDocs + Material、MyST (reStructuredText 的 Markdown 方言)、Read the Docs 发布流程——一个正规开源项目的文档管线。

·typing 与文档的融合：PEP 484 类型注解 + PEP 257 docstring + sphinx-autodoc-typehints, 让"签名、类型、说明"三位一体。现代大型项目 (如 pandas) 已全面采用。

·sys.modules 与导入副作用：理解"导入即执行"模型 (Pydoc 的运行基础), 以及 if name == 'main' 守卫的工程意义——这是所有"可导入又可运行"模块的规范写法。

·doctest / pytest --doctest-modules: 把 docstring 示例纳入自动化测试套件——文档漂移 (doc rot) 的天敌。

本章学习建议 (Learning Advice)

·重要程度：★★★★☆ (4/5)。本章没有新语法, 但 dir() / help() / docstring 是每天都要用的三件套——它们在后面所有章节 (尤其学函数、类时) 都会持续发挥作用; "常见编码陷阱"一节则是对第三部分语法的一次"疫苗"。

·应该掌握到什么程度:

·必会 (形成肌肉记忆): 给模块/函数/类写三引号 docstr

ing; 用 `dir()` 查属性; 用 `help()` 查任何对象; 用 `print(obj.doc)` 查看文档; 记住"原地修改的方法返回 `None`"与"调用函数必须带括号"。

·应知: `docstring` 的语法规则 (必须是组件体的第一个语句、`f-string/bytes` 除外、`\U` 转义与 `raw` 字符串的边界); `Pydoc` 的 `-b` 浏览器模式与 `-w` 导出; `help('模块名')` 字符串式调用与 `help('xyz')` 的坑。

·了解: `pydoc.css` 定制、`Sphinx` 是什么、官方手册两大分区 (`Library vs Language Reference`)、"What's new"文档的用途。

·不要求: 背下 `Pydoc` 全部命令行选项; 掌握 `CSS` 定制; 现在就学 `reStructuredText`。

·后续学习内容:

·立即: 第 16 章"函数基础"——写第一个带 `docstring` 的函数, 并用 `help()` 验证它; 第 3 部分练习 5 的 `map/lambd` 将在第 19 章回响。

·近程: 第五部分 (模块) 将解释 `import` 机制、`name` 守卫、`sys.path`——它们是理解 `Pydoc` 行为 (搜索路径、导入即执行) 的底层知识。

·进阶 (学完本书后): `inspect` 模块、`functools.wraps` 与装饰器文档、`Sphinx` 建站、`doctest` 接入 `CI`。

·实践建议:

1. 现在就在 `REPL` 里跑一遍本章所有命令: `dir(sys)`、三个长度统计、`help(sys)`、`help(str.replace)`、`help('sys')`, 亲眼看数字 (你的 `Python` 版本可能与书里不同)。

2. 为本书前 14 章写过的任何一个脚本补上模块 docstring 和函数 docstring, 然后 `help()` 它——体验“文档从对象身上长出来”。
 3. 运行 `python3 -m pydoc -b`, 在浏览器里找到你的模块——顺便点开 `sys` 的页面看看标准库文档长什么样。
 4. 把“常见编码陷阱”十一条抄成一张卡片贴在手边——未来两周写代码时对照自查。
-

本章完。下一部分：Part IV, Functions and Generators (函数与生成器) ——详见第 16 章。